

1 Article

2 **Prediction-Driven Non-Preemptive Scheduling with Bounded**
3 **Overtaking for Shared Execution Services under**
4 **Deadline-Driven Bursty Load**5 [First Author Name]¹ and [Second Author Name]^{2,*}6 ¹ [First author: department, institution, city and postcode, country]; [first author e-mail]7 ² [Second author: department, institution, city and postcode, country]; [second author e-mail]

8 * Correspondence: [corresponding author e-mail]

9 **Abstract**

Automatic graders for programming courses, build farms and serverless platforms run other people's jobs on k servers without interruption, each job under a time limit L . Ordering the queue by predicted running time shortens most waits, but one wrong prediction can leave one job far behind. We ask what such a service can promise to every job when the predictions are arbitrary. We prove that under any non-preemptive work-conserving policy, on every arrival sequence, a job's delay relative to first-come first-served (FCFS) equals the net work that overtook it divided by k , to within $(2 - 2/k)L$, and the constant is exact. A budget on overtaken work is therefore sufficient for a per-job guarantee and necessary up to an additive $O(kL)$; a count of overtakes is not. A wrapper that enforces the budget bounds every job's wait against FCFS for arbitrary arrivals, predictions and base policies, with the exact constant $(3 - 2/k)L$. We evaluate on logs from two judging platforms, a serverless trace, two continuous-integration pools and a compute farm. With scores that use only results available in the replay, a 600-second guard closes 0.672 of the FCFS-to-shortest-job-first tail-wait gap at the busiest development load, interval [0.602, 0.743].

Keywords: non-preemptive scheduling; scheduling with predictions; multiserver queues; bounded overtaking; per-job guarantee; first-come first-served; runtime prediction

MSC: 90B35; 68M20; 60K25; 68W27

1. Introduction

Before a common deadline, submissions that looped until their resource limit took graders out of service, and the waits of everything queued behind them grew in a cascade. The maintainers of a university autograder found this when they replayed their busiest four hours [1]. A contest judging system documents the same problem from the other side: its manual recommends one judging host per 20 teams and warns that the waiting time grows with fewer hosts [2]. Both are shared execution services. Such a service runs jobs for many users on a fixed pool of k servers, and a job that has started is not interrupted: it runs until it finishes or until it reaches a per-job time limit L . Serverless function platforms, build and test farms for continuous integration, batch inference queues and the judging back ends of programming-assignment platforms [3] are all organised this way.

Two properties of the load make such a service slow down exactly when it is used most. Arrivals follow an external calendar, so they are neither stationary nor independent;

Academic Editor: Firstname
Lastname

Received: date

Revised: date

Accepted: date

Published: date

Citation: To be added by editorial staff
during production.

Copyright: © 2026 by the authors.

Submitted for possible open access

publication under the terms and

conditions of the Creative Commons

Attribution (CC BY) license

([https://creativecommons.org/](https://creativecommons.org/licenses/by/4.0/)

[licenses/by/4.0/](https://creativecommons.org/licenses/by/4.0/)).

the same crowding before a fixed cutoff has been measured directly [4–6]. And a small fraction of the jobs carries most of the work. On the serverless invocation trace we use [7,8], the longest 1% of invocations account for 44% of the total work. On the main judging trace, after the analysis service cap is applied, the longest 1% still account for 37.5%, and for 27.5% on the second. A few very long jobs therefore sit in front of many short ones when the queue is longest.

The mechanism we exploit is visible in a queue of three jobs on one server (Figure 1). Job *A* loops forever and is stopped by the 60-second limit; jobs *B* and *C* take 0.2 seconds each. In arrival order the mean wait is 40.1 seconds. Shortest first, it is 0.2 seconds, at a cost of 0.4 seconds to *A*. The server does the same total work in both orders. Nothing has been simulated yet. The three numbers are arithmetic, and they rest on one assumption: that the scheduler knows, before *A* starts, that *A* will be long. That assumption raises three questions, and the paper is organised around them. Can the cost of a job be estimated when it arrives, from history that is visible at that instant? Given estimates, what should a non-preemptive scheduler rank by? And what happens to one job when the estimates are wrong?

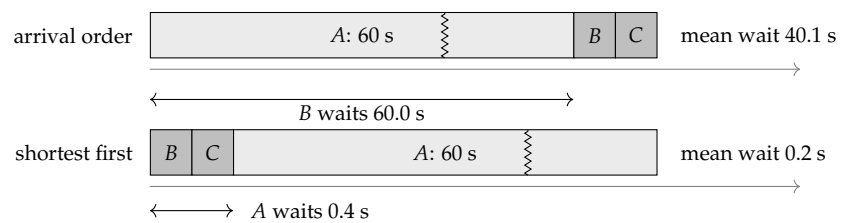


Figure 1. The three-job example of Section 1 on one server: *A* is stopped at the per-job limit $L = 60$ s, *B* and *C* take 0.2 s each, and all three are present at $t = 0$. The boxes are schematic, not to scale, and the zigzag marks the elided part of *A*. The two orders execute the same total work; reordering moves 60 seconds of waiting off *B* and *C* and puts 0.4 seconds onto *A*. Source: the arithmetic stated in the text, not a measurement.

The third question is the one an operator asks first, because it is the one a student will ask. Ordering by prediction improves the bulk of the distribution and can damage the tail: in our busiest configuration one job waited almost a hundred job-limits longer than it would have under first-come first-served (Section 8.5). The scheduling-with-predictions literature bounds an expectation against an optimal or shortest-remaining-processing-time reference, and its constant-factor results assume that the prediction error is bounded multiplicatively [9,10]. No one can promise that about a deployed predictor. There are results on single axes. One line bounds the maximum stretch over all jobs on a preemptive machine, which is not an expectation [11]. Another bounds a request's latency in a stationary light-tailed queue against the workload the system holds at the request's boosted arrival instant, using the work that overtakes it [12,13]; read on a sample path, that is close to a bound against first-come first-served, but it is not the form those papers state. What we do not find is the combination: a first-come first-served reference, stated per job, on an arbitrary arrival sequence with arbitrary predictions, without preemption, on k servers, and enforced online around whatever policy is underneath rather than analysed for one fixed policy.

We change the reference point instead of strengthening the assumption. For each job separately, we bound how much longer it waits than it would have waited under first-come first-served on the same arrival sequence, assuming nothing about the arrival process and nothing about prediction quality. The mechanism is an overtake budget. For every waiting job we account the total work of later-ordered jobs that completed while it waited, and at each dispatch decision the earliest-ranked waiting job whose account has reached its budget takes precedence. The guard wraps an arbitrary base scheduler,

including a learned one, and touches only the dispatch decision. One parameter matters for the promise, a cap $B_{\max}$ on the budget. What the budget does below that cap is a design freedom: it changes how much of the base policy's benefit survives and which jobs bear the harm, and it changes the promise not at all.

We budget work rather than count overtakes because an identity puts the guarantee in units of work. A job's delay relative to first-come first-served equals its net overtaken work divided by k , to within $(2 - 2/k)L$, for every non-preemptive work-conserving k -server policy and every arrival sequence. The identity does not refer to the policy, and it holds in both directions, which is what makes a work budget the right unit and a bound on the *number* of overtakes the wrong one. The guarantee is measured in units of L , so it is most informative when L is comparable to the waits an operator cares about. A computing farm whose jobs run for days is a case where the certificate is too coarse for those waits; that does not rule out tighter guarantees for a restricted policy or workload.

On the first two questions, arrival-time history is enough to tell the heavy jobs from the rest on all three platforms. We compare three families of predictor under one split and one visibility protocol: tabular models [14], a sequence model over each user's history, and a heterogeneous graph neural network over the user, task and assignment entities [15]. The graph model does not improve on the tabular one. Removing every edge leaves its classification scores where they were, and so does permuting the edges at random, so the signal is in each entity's own history rather than in the relational structure. The edges' contribution to the rank correlation is small over all rows, 0.006 over the permuted-edge control, and larger on tasks with fewer than five prior results, where the graph model exceeds that control by 0.028 and the tabular model by 0.146. We report that as a property of this prediction task and not of relational models (Section 4). We rank by the estimated conditional expected cost $\mathbb{E}[C \mid x]$ rather than by a log-scale prediction, whose back-transform generally differs from the arithmetic mean; the single-server interchange argument motivates that choice without establishing optimality for several servers, arbitrary arrivals or deadline-window percentiles. Errors in the fitted score are asymmetric: treating a heavy job as light, and letting it run first, is the expensive mistake by about an order of magnitude (Sections 5 and 8.4).

We make three contributions.

- We prove the net-overtake identity, show that its constant $(2 - 2/k)L$ is, for $k \geq 2$, an exact supremum, approached and never attained, and prove the corollary in the other direction: any per-job guarantee of this kind bounds the overtaken work, up to an additive $O(kL)$.
- We give an overtake-budget guard that wraps any base scheduler and bounds every job's wait against first-come first-served, for k servers, arbitrary arrivals and arbitrary predictions. The bound reads the budget only through its cap, so the operator sets $B_{\max} = k(G - (3 - 2/k)L)$ from a promise G in seconds, with no data. The additive constant $(3 - 2/k)L$ is an exact supremum, never attained, and it grows with k from L towards $3L$. One budget shape, whose allowance grows with the wait a job has already had, carries a multiplicative bound as well. A maximum waiting time, which serves the earliest-arrived waiting job once it has waited a fixed time, makes a promise of the same form; at the busiest load and promises of five to ten job-limits it closes 0.03 to 0.42 of the gap where the guard closes 0.48 to 0.80, both under the source platform's result clock, because it charges a job for the backlog ahead of it. From the arrival log alone we also bound the absolute wait, so an operator can promise a number of seconds rather than a margin over arrival order; quoting it in advance needs a forecast of the arrival burst, which on our traces needed a margin of about a third.

We state which assumptions can be dropped and show by counterexample that work conservation cannot.

- We evaluate on real logs from two programming-assignment judging platforms [16,17] and a serverless invocation trace [7], with every simulated job checked against the bound, and we compare cost predictors on causally visible history, with two conclusions that carry into the scheduler: expected-cost ordering performs better than the log-scale alternative in the reported source-clock comparisons, and the fitted score is more sensitive to severe underestimates. Two applicability conditions, read off six workloads [18–21], say where the method does not apply, and most of the six fail one of them.

Three qualifications hold over everything reported below. The judging logs record execution demand and do not identify a historical shared queue, so the pooled capacity and the replayed waiting times are constructions of ours (Section 7); a capacity sweep, a timed-work service, a scripted two-worker run on a course platform’s back end and a closed-loop replay test how far the results depend on that construction (Sections 8.9 and 9). Every figure in this paper is a development figure: the main trace’s three sealed terms will be opened once, after the method, the parameters and the primary metric have been frozen, and their numbers reported separately, while the sealed parts of the other two traces stay shut (Section 7.6). And the empirical headline uses scores that use only results available in the replay: each policy’s replay is repeated, leaving out every result that was not yet complete when the job that uses it arrived, until no score uses an unavailable result. Under that protocol Guard(600) closes 0.672 of the gap between first-come first-served and shortest-job-first at the busiest load, with interval $[0.602, 0.743]$, against 0.801 under the source platform’s own result clock, which we keep as an optimistic reference (Section 8.3).

Section 2 places the paper in the literature, and Section 3 states the model, the visibility protocol and the metrics. Sections 4 and 5 give the cost predictors and the scheduler, Section 6 the identity and the per-job bounds, and Section 7 the traces. Section 8 reports the experiments, Section 9 the limitations, and Section 10 what an operator can now promise. The deferred proofs and the tightness constructions are Supplementary Sections S4 and S8.

2. Related Work

What an operator would need to quote to a student is a bound on one submission’s wait. Four lines of work come close to stating one, and each stops short in a different place.

2.1. Scheduling with Predictions

Purohit, Svitkina and Kumar [22] frame predictions in online algorithms as a trade-off between consistency and robustness. Predicted-size scheduling has since been studied through queueing models in which the prediction is a random variable [23], bounded multiplicative error [9], worst-case distortion [24,25], many-server limits [26], prediction acquisition [27] and per-job stretch on a preemptive machine [11]. The survey of Mitzenmacher and Shahout [10] brings this line together. Our reference is FCFS rather than an optimum, our scope is one job rather than a mean, arrivals and prediction errors are arbitrary, and service is non-preemptive. The bound we get is limited in the same measure: it controls extra waiting relative to FCFS and does not assert an improvement. Whether the guard is useful is an empirical question. It depends jointly on size dispersion, predictability and arrival structure, not on membership in a heavy-tail class.

2.2. Fairness Relative to FCFS, and Bounded Overtaking

Worst-case per-job fairness is older than fair-start-time metrics. Schwiegelshohn and Yahyapour [28] call a schedule λ -fair when jobs submitted after job i cannot increase i 's flow time by more than a factor λ , and they bound fairness and efficiency together for a preemptive gang policy. Their bound is multiplicative, and for a short job it is tighter than one measured in L ; gang preemption is what makes it possible. Our guarantee is additive, non-preemptive and referenced to the same input under FCFS. Section 6 shows only that a wrapper of our family, one that protects against every opaque base proposal sequence, cannot promise less than order L once it permits overtaking; that is not an impossibility for all scheduling mechanisms. Supplementary Section S9 gives their constants and the per-paper detail this section compresses.

Fair-start-time work gives a closer reference. A job is compared with a re-simulation that excludes later arrivals [29], and the excess is then used as a job-level metric [30,31]. Conservative backfilling instead fixes a reservation for every queued job, so later backfills cannot delay that promised start [32,33]; it is a $G = 0$ endpoint relative to the reservation, not relative to same-input FCFS. Slack-based backfilling gives each job a slack at submission and lets later jobs delay it only within that slack, which bounds every job's wait against its own reservation [34]. PV-EASY [35,36] enforces zero lower-priority delay by killing and restarting the backfilled jobs that stand in the way. These guarantees are on-line, but they use different references or preemption. Ours limits the additive FCFS excess of a non-preemptive wrapper.

The closest architectural precedent for a wrapper around an arbitrary policy is Load Balancing Guardrails [37]. It augments any dispatching policy so that a load-balancing system with SRPT at each server is optimal for mean response time in heavy traffic. Its reference, its stochastic objective, its preemptive server policy and its dispatch-level constraints all differ from our per-job additive promise, but the separation between an unrestricted base decision and a protective wrapper is the same. At one server our "pointwise maximal" statement uses the supervisory-control meaning of a maximally permissive supervisor, which turns off only the actions needed to keep behaviour within the specification [38]; we claim that property only for the opaque-base, integer-time class defined in Supplementary Section S8, not as a general least-restrictive result. Supplementary Table S24 compares the reference discipline, arrivals, preemption, information, server model and guarantee across these nearest mechanisms.

Bounded overtaking has a history in queueing theory, in two units. Counting came first. Nudge lets a small arriving job swap past the job at the tail of the queue [39], and Nudge- K lets it pass up to K others while no job is passed more than once [40]; the line continues with Nudge*(M) [41]. The finite-skip models of WCFS restrict service to the oldest n jobs and add work conservation and non-idling [42]. Work is the second unit: γ -Boost measures the *crossing work* of later arrivals that precede a tagged job [12], and that line extends to k servers [13]. All of these are analysed in steady state, under Poisson arrivals with light-tailed or phase-type sizes, and they target stationary means or tail constants rather than a deterministic per-job cap enforced on every sample path. That Nudge implies, on every sample path, a per-job additive bound of one small-job length is our own reading and not a claim of [39], whose analysis is stochastic by design. We do not claim bounded overtaking at one server as new.

The additive shape and the constants are familiar on their own. Additive bounds against a fluid or ideal reference are the standard form in packet scheduling, one-sided in most analyses [43–45] and two-sided in WF²Q and EEVDF [46,47], and the coefficient $2 - 1/k$ of Graham's list-scheduling bound has the shape of the $(2 - 2/k)$ and $(3 - 2/k)$ above [48]. What we prove is not a conservation law in the sense of Kleinrock [49]. Supple-

mentary Section S9 states each of these bounds in the form its source gives it, and carries the rest of this line.

2.3. Runtime Prediction and Scheduling in Deployed Systems

Backfilling schedulers on high-performance computing systems have used predicted runtimes in place of user estimates for two decades [50], with predictions learned from past jobs and queue reordering tuned online [51,52]. Their jobs each hold several processors for a requested time, and the metrics they report are means, of bounded slowdown [51] and of waiting time [52]. Closest to our setting, Zhang, Wu and Lu [53] predict the running time of submitted code in a judging queue and dispatch in increasing predicted runtime inside fixed-size windows, so we are not the first to schedule a judging queue by predicted cost. A fixed window does limit overtaking, but by position, and its abstract reports no per-job bound and no test under adversarial predictions. In inference serving, where non-preemption is close to a hard constraint, recent systems learn a relative order instead of a length [54] or target the tail because prediction-driven policies are fragile under distribution shift [55]. Production systems otherwise avoid starvation with aging counters and maximum waiting times, and recent inference servers promote a request once its wait passes a fixed threshold [56] or a multiple of its service time [57], none of them with a stated bound. A maximum waiting time that serves the earliest-arrived waiting job once it has waited a fixed time does carry a per-job bound against FCFS of the same form as ours (Section 6.7); at the busiest load and tight promises it keeps much less of the benefit (Section 8.5). Serverless platforms are in the same position: SFS approximates shortest-remaining-time-first from user space [58], and ALPS learns a per-function rank and time-slice cap from an offline replay [59], neither with a bound. Our guard is a wrapper rather than a policy, so it can be placed around any of these schedulers, and its bound does not depend on which one is underneath.

2.4. Time-Varying and Deadline-Driven Load

That people concentrate their actions near a cutoff has been measured directly, in time-management research [4] and in conference registration data [5,6]. The queueing consequences of an arrival rate that varies on the timescale of the service time are also well studied. Peak congestion in a multiserver system lags the peak of the arrival rate [60], and staffing rules that follow a time-varying rate rest on offered-load approximations [61,62] or on simulation [63] rather than on stationary formulas. These results say how many servers to provide. They give no waiting bound for an individual job and no guidance for a queue that has already formed, and we leave capacity fixed. The steady-state classified-priority formula that motivates ranking by predicted cost [23,64] does not hold up under this load either; Section 3 measures the mismatch, which is why we state the guarantee on sample paths and assume nothing about arrivals.

2.5. Relational and Tabular Models for Cost Prediction

Our comparison includes a heterogeneous graph neural network over users, tasks and assignments, in the relational graph convolution family [15], with the neighbour aggregation chosen on validation between the mean and the sum [65]. Two features of the published benchmarks make them a poor argument either way. In RelBench [66] and its successor [67] the gradient-boosted baseline is fitted on the raw columns of the entity table alone, without hand-built aggregates over related tables, so the margin reported there is not a comparison against a tabular model that carries engineered history features. The same paper's study against a human data scientist, which is the closer comparison, records regression as the region where the relational model struggles. Cost prediction is a regression task on entities with long individual histories, which places it in that region,

and the evidence that tree ensembles remain strong on tabular data concerns single tables [68]. We therefore settle the question inside our own protocol instead of importing a conclusion: a tabular baseline that carries engineered causal history features [14], an ablation ladder that removes and then permutes the edges, and a comparison made on queueing outcomes rather than on AUROC alone. The negative result we report is about this task on these traces, where each user and each task already has enough history of its own. The relational signal we find is small overall and concentrated on tasks with fewer than five prior results.

2.6. What Is Missing

Our contribution is therefore a combination and not a new unit. The identity is two-sided and holds on an arbitrary arrival sequence with arbitrary service times, under no distributional assumption. The constant $(2 - 2/k)L$ is exact for k non-preemptive servers, and so is the $(3 - 2/k)L$ of the wrapper. The converse makes a work budget necessary and not merely sufficient, up to an additive $O(kL)$. And Algorithm 1 enforces the bound online around any base policy, learned ones included, under predictions that may be arbitrarily wrong. On a specified batch family the bound is also achievable, and at one server the wrapper is pointwise maximal within an opaque-base class (Supplementary Section S8); neither result claims global instance optimality or a least-restrictive wrapper for $k \geq 2$.

3. Problem Model

3.1. Jobs, Servers and Dispatch

The judging queue of the introduction becomes a finite set of n jobs served by k identical servers. We use *server* throughout for what a deployment would call an executor, a judge host or a worker, and *job* for what a platform would call a submission, an invocation or a run. Job i arrives at time $a_i \geq 0$ and requires C_i units of service. Service is non-preemptive, so a server that starts a job keeps it until the job finishes or until it has been running for the per-job time limit L , at which point the system stops it. Throughout, C_i denotes executed work, so $0 < C_i \leq L$ by construction and a job stopped at the limit contributes exactly L . An enforced timeout ends such a job without a successful result; the waiting-time guarantee does not bound that loss of completion quality. Observed maxima and analysis truncations in a trace are distinguished from enforced timeouts in Section 7. One quantity carries three names in this paper: we write *cost* in the application sections, *service* or *service time* in the analysis, and *work* when several jobs are summed, and all three denote C_i . Two further words are fixed here because the application sections need them. A *task* is the exercise or problem a job is an attempt at, so many jobs share one task; a *run* is the unit of work of a continuous-integration pool, and is a job in the sense above.

Servers are work-conserving, meaning that no server idles at an instant at which some job waits. Jobs carry a rank, the total order induced by the pair (arrival time, input index); we write $j < i$ for “ j has the smaller rank”, which implies $a_j \leq a_i$. Events occurring at a common instant are processed as completions, then arrivals, then dispatches. A policy is any rule that names one waiting job and dispatches it, whenever a server is free and at least one job waits. It may be randomised, learned, priority-driven, or fed by predictions of arbitrary quality. We assume nothing else about it.

Arrival times are arbitrary. We place no stochastic assumption on a_i , on C_i , or on the dependence between them: an external calendar generates the load we care about, and that load is neither stationary nor Poisson.

3.2. What the Scheduler Knows

The service time C_i is hidden at arrival. The scheduler reads it at the instant i completes and not before. Any service that runs submitted work without looking inside it first is in that position. Available at a_i are a feature vector x_i and a score $\hat{C}_i$ computed from it.

Visibility is causal, and we define it by result availability instead of timestamp order. Write done_j for the instant at which the outcome of job j became readable and $\delta \geq 0$ for the delay with which outcomes reach the feature store. Then

$$\text{information about } j \text{ may enter } x_i \iff \text{done}_j + \delta \leq a_i.$$

Historical features are recomputed under the chosen release rule rather than assembled once and masked afterwards. The original prediction study in Section 4 uses the source platform's result clock. In a replayed queue, an endogenous result instead becomes available at $\text{done}_j^P = a_j + W_P[j] + C_j$. Source-clock causality therefore does not imply availability under policy P : a recorded result may already be visible while its replayed job is still waiting. The headline of Section 8.3 therefore uses an exact protocol: each policy's replay is repeated, leaving out every result that the replay says was not yet complete when the job that uses it arrived, until no score uses an unavailable result. Two references sit beside it. A conservative protocol restricts history to replayed submissions in the same class-term copy, one class of one term as placed in the overlay, and delays every result by a fixed lag; a static control uses no outcome history. Section 8.3 says how availability is checked and separates external history from queueing exposure inside the same copy. The guard theorem does not depend on which scores these protocols produce.

3.3. Waiting Time, the FCFS Reference and the Service Guarantee

Let s_i^P be the start time of job i under policy P and $W_P[i] = s_i^P - a_i$ its waiting time, which excludes service. FCFS is the policy that always dispatches the waiting job of smallest rank, and that dispatches in increasing rank when several servers become free at the same instant. We run it on the same arrival sequence, with the same service times and the same k , so that the comparison is pathwise rather than distributional. The per-job excess is

$$\text{excess}_P[i] = W_P[i] - W_{\text{FCFS}}[i].$$

Two further quantities carry the analysis. Writing $j \prec i$ for " j is dispatched before i " in the dispatch sequence,

$$\text{In}_i = \sum_{j \succ i, j \prec i} C_j, \quad \text{Out}_i = \sum_{j \prec i, i \prec j} C_j,$$

the work that overtook i and the work that i overtook. Section 6 relates them to $\text{excess}_P[i]$.

We say that a policy P offers a service guarantee G if $\text{excess}_P[i] \leq G$ for every job i , on every arrival sequence, for every vector of service times bounded by L , and for every score, adversarial scores included. Two words are kept apart from here on: the *promise* is the number G the operator states, and the *guarantee* is the proved statement that $\text{excess}_P[i] \leq G$ holds for every job. An operator chooses a promise; a theorem supplies a guarantee. The statement is per job and holds on the realised sample path, in no sense in distribution or on average, and it does not assume that the score is any good. Packet scheduling states per-individual additive bounds against a reference discipline [43], and batch schedulers measure the same difference after the fact as a fairness diagnostic [29,30]. Those authors define the fair start time by re-simulating the scheduler under test without the later arrivals, in two variants, and in the relaxed one the job may not start before any job queued ahead of it. Our reference runs first-come first-served for every job in-

stead, so the two coincide when the scheduler under test is first-come first-served. The scheduling-with-predictions literature does something different: it compares against an optimal policy, with a competitive ratio that degrades with the prediction error [22] or an average-case ratio under a bound on that error [9].

We take FCFS as the reference because it is what these services deploy by default, so the excess is a quantity an operator can be held to. It also has a known extremal property for the unfinished-work vector in a multiserver system: the FCFS workload vector is minimal in the weak-majorisation order among systems whose assignment of arrivals to servers does not depend on service times [69], a hypothesis narrower than work conservation alone. That extremality is why single-server arguments do not transfer unchanged to $k > 1$.

3.4. Metrics

The primary metric is the 99th percentile of waiting time inside deadline windows, and we write $p99_{dl}$ for it, the symbol the tables carry as a column head. A deadline window is the 24 hours before a deadline; a job belongs to the window if and only if its arrival falls inside it, and a job that arrives inside but completes after the window closes is still counted. We fix this choice before any sealed part of a trace is opened.

Four secondary quantities go with it. The first is the mean wait over all jobs. The second is the maximum of $excess_p[i]$, the quantity a service guarantee bounds. The third is the *harm*: the largest excess over FCFS among the jobs FCFS would have started within 1 s, that is, among the jobs FCFS would not have delayed at all. A guarantee G says nothing about how that excess is distributed, and the harm is what an operator's users complain about. The fourth is the *firing rate*, the share of dispatch epochs at which the guard of Section 5 overrides the base policy, weighted by queue length. We also report the worst wait among heavy jobs, heavy being the label Section 4 defines.

Improvements are also expressed as the fraction of the gap closed,

$$\frac{M_{FCFS} - M_P}{M_{FCFS} - M_{SJF}},$$

where M_Q is the primary metric, $p99_{dl}$, under policy Q , and SJF sorts waiting jobs by their true service times. SJF on true sizes is a reference policy and not an optimum. It is not deployable, since C_i is unknown at arrival, and it does not minimise any of the metrics above on a non-preemptive multiserver system. The ratio is unstable when its denominator is small, so we mark such configurations and print the two absolute waits next to every ratio. The fraction of the gap closed is not a percentage reduction in waiting time and we do not report it as one.

3.5. Steady-State Models and This Load

Stationary queueing models give closed forms for the mean wait of a non-preemptive priority class [64, pp. 499–507] and would let class thresholds be placed analytically. They need a stationary arrival process, which deadline-driven load does not supply. On a synthetic Poisson stream the formula agrees with simulation to 0.4%, but on the same workload at its recorded timestamps the measured wait is 10.7–13.7 times what it predicts (Supplementary Section S2.5). We therefore use steady-state expressions as motivation only, and every promise here is a per-job statement on the realised sample path (Section 6). The mechanism that makes reordering useful is the three-job arithmetic of the introduction, where total work is unchanged while one long job moves back a little and several short ones move forward a lot.

4. Cost Prediction

Before the looping submission of the introduction starts, something has to decide that it will be long, from history alone and at the instant it arrives. That decision is a number, and the scheduler of Section 5 reads it. We say here what the number estimates, how we keep its inputs causally visible, which model families we compared under that one protocol, and which model we send into the queue. All figures are development figures, and the traces they come from are described in Section 7.

4.1. Two Targets

A scheduler and an evaluation want different things from the same model. The scheduler wants the conditional mean $\mathbb{E}[C_i | x_i]$, which is the single-server pairwise-interchange target and the candidate score fitted in Section 5.1. That section also explains why the interchange argument does not establish optimality on several servers, where we evaluate the order empirically. For comparing predictors we also need a target that stays interpretable when the mix of jobs shifts between periods, so we score each model on a binary label. A job is *heavy* when its executed cost C_i exceeds the 95th percentile of the training rows. This is the one definition of heavy used in the paper, and the worst heavy-job wait of Section 3 is taken over the jobs it selects. On the main judging trace the cut-off is 1.559 s and heavy jobs are 1.31% of the test period; on the second judging trace it is 1,314 ms and 5.07%. The label is fixed on training rows and never refit on the period being scored. On the main trace its rate in the test period is about a quarter of its training rate, 1.31% against 5.00%; on the second it stays at about the training rate. Because the positive class is small we report average precision next to AUROC. For the regression output we report RMSE on $\log(1 + C)$ and Spearman correlation, since a ranking policy consumes order rather than level.

4.2. The Visibility Protocol, Made Operational

Section 3 defines visibility by result availability: information about job j may enter x_i exactly when $\text{done}_j + \delta \leq a_i$. Turning that into a feature pipeline takes three commitments.

First, every historical feature is recomputed by streaming the log in availability order and reading each entity's state as of the query instant. No feature is assembled over the whole period and masked afterwards. We test this by recomputing the features from scratch and comparing: 0 rows disagree in any of the eight configurations. Supplementary Section S1 gives the audit in full, including the same test applied to the natural but wrong construction and the separate checks on the graph pipeline.

Second, thresholds are refit per target period. We recompute the heavy cut-off and the class boundaries used inside the models from the training rows of that target, and we predict each target period with a model fitted only on periods before it and frozen before the target's first arrival. Training rows whose results would not have been available by that instant are dropped; on the primary configuration the rule binds on 0 rows. We report the zero because a rule that never binds is easily mistaken for a rule that is not there.

Third, we treat the delay δ with which results reach the feature store as a sensitivity, not as an assumption. Supplementary Section S1 gives the two clocks in full. Under them, ranking survives a ten-minute feedback delay while the probability calibration gets worse.

All families see the same rows, the same split and the same availability rule. Supplementary Section S1 defines the three families and their ablation ladders: the gradient-boosted tabular groups M1 to M5, which add task history, user history, static features of the submitted artefact and one-hop relational aggregates in turn; the sequence family R1 and R1S, a GRU over the user's own earlier jobs, used alone and as a state vector inside the boosted model; and the relational ladder N0, G0U, G0, G1 and G2, a heterogeneous

network that starts from a multilayer perceptron on M4's columns and adds empty neighbour slots, then the real edges, then edges permuted inside a period and time block. Two entries are controls rather than models. M6 scrambles each job's neighbourhood, and G2 permutes the edges. If the relational structure carries signal, these two must lose it.

4.3. What the Comparison Shows

Supplementary Table S2 gives the two judging traces. One-hop relational aggregates add nothing on either: M5 – M4 is +0.0019 of AUROC on the first and –0.0002 on the second. The ladder gives the same result with a trained model in place of hand-built aggregates. Deleting every edge (G0) and permuting the edges (G2) leave AUROC within 0.002 of G1, and the paired intervals for G1 – G0 (–0.0001 [–0.0019, 0.0018]) and G1 – G2 (+0.0019 [–0.0000, 0.0042]) cover zero. The one step of the ladder that resolves is N0 → G0U, +0.0139 [+0.0090, +0.0195] of AUROC at identical information. What moves there is the encoder, with the edge set held empty in both.

Edges do carry something, and it is confined to cold-start tasks. On the 1,092 test rows whose task has fewer than five visible prior results, G1 beats M4 by +0.1464 [+0.0997, +0.1887] of Spearman correlation, and beats its own permuted-edge control by +0.0282 [+0.0130, +0.0432]. Over all rows the G1 – G2 Spearman difference is +0.0063 [+0.0014, +0.0111], small and resolved; the corresponding AUROC and AUPRC differences are not. We read this as a task property rather than a verdict on relational models. When each entity's own recent record is visible and rich, a neighbourhood adds little, and the place where it adds anything is the entity that has no record yet.

The sequence state has the best prediction scores of anything we fitted, and no consistent effect in the queue. R1S reaches AUROC 0.9366 against M4's 0.9283, a difference of +0.0083 [+0.0038, +0.0131], and it also wins on RMSE and Spearman. Inside the queue we find no consistent improvement. Against the same ranking objective it gains +0.007 [0.003, 0.011] of the deadline-window gap at busy-hour utilisation 0.8, loses –0.010 [–0.022, –0.002] at 0.5, and is open at 1.0. A control fitted on the same 90% of rows that the GRU's early stopping left available moves the same way, so the smaller training set does not explain it.

Which information matters is not a constant across platforms: the user's own history leads on the first judging trace, and the task's history leads on the second by 0.11 of AUROC (Supplementary Table S2). The four non-judging traces spread wider still, and Supplementary Section S1.2 reads them by feature group, the AUROC column of Table 7 carrying the figure the scheduling results are read against.

4.4. Cost of Predicting, and What Enters the Scheduler

Prediction has to be cheap enough to sit in front of a queue whose median job is 204 ms. It is. Charging the measured per-job cost of each family to the predictive policies before they enqueue changes the closed gap by less than 0.001 at every load we ran (Supplementary Section S1).

Which model enters the queue is a queueing decision, not a prediction decision. M5 has the higher AUROC of the two tabular candidates and loses in the queue at all three loads, by 0.012, 0.008 and 0.010 of the deadline-window gap, with intervals excluding zero in each case. R1S has the higher AUROC of everything we fitted and does not win consistently. We therefore take M4, the full tabular model, into Section 5. We selected it on the queueing outcome on validation periods, not on prediction quality. The best predictor was not the best scheduler, and the two orderings do not agree.

5. Scheduling Algorithm

The scheduler has two layers, because the ranking that shortens the queue is also what can leave one submission behind without end. A base policy orders the queue by a score computed at arrival, and a wrapper limits how much work may overtake any waiting job. The two are independent: the wrapper accepts any base policy, and its guarantee does not depend on that policy behaving well.

5.1. Ranking by Expected Cost

When a server frees, the base policy takes the waiting job with the smallest score. For a batch on one server, the pairwise-interchange argument orders jobs by their conditional mean service times. The same conclusion does not hold in general on $k \geq 2$ servers. For example, on two servers let $A = 2$, $C = 3$, and let B be 0.1 with probability 0.9 and 100 with probability 0.1. Ordering the three conditional means starts A and C and has expected mean wait $2/3$; starting A and B instead has expected mean wait 0.096667. Expected-cost ordering is therefore an empirically motivated base policy here, not a multiserver optimality claim.

Both raw-scale squared error and the Tweedie mean deviance have the conditional mean $\mathbb{E}[C_i \mid x_i]$ as their unrestricted population optimum. Their fitted orders can nevertheless differ with finite model capacity, the link function, regularisation, sample weighting and optimisation. We therefore compare objectives empirically while keeping the features, model size and rolling-origin protocol of Section 4 fixed. SPJF-E uses raw seconds with a Tweedie loss of power 1.5, which accommodates mass near zero and a long right tail [70,71]; SPJF-log uses the same pipeline fitted by squared error on $\log(1 + C)$, and ranks by that fitted value, which orders jobs as its inverse would.

The literal inverse $\exp(\mathbb{E}[\log(1 + C) \mid x]) - 1$ is no larger than the conditional arithmetic mean, with equality only in the degenerate case. The gap is not determined by the conditional variance alone. To use a log-scale regression as a mean estimator under its stated conditions, one needs a residual-distribution correction such as the smearing estimator of Duan [72].

Only the order of the fitted score is used. Its level does not transfer across periods, since the share of heavy jobs in the training periods differs from the target period by a factor of several, so we never read the score as a quantity, neither as an estimated finish time nor as a threshold. The scheduler is therefore invariant to a monotone recalibration of the score.

Section 8.4 measures what the fitted objective gains, together with the alternatives we tried and rejected: a predicted quantile, a two-class rule, and squared error on raw seconds. Its partial-oracle repair replaces selected scores with truth and is not invariant to an arbitrary monotone rescaling of the original output. It therefore identifies the errors of this fitted predictor; it does not establish a universal asymmetry of prediction errors or a different population estimand.

No choice of ranking function bounds the worst case. A job the score gets wrong can still be passed without end, whatever the score is and however it was fitted; Section 5.2 is what bounds it.

5.2. The Overtake-Budget Guard

Fix a base policy A . For every waiting job q and every time t define

$$\text{over}[q](t) = \sum \{ C_j : j > q \text{ in rank, } j \text{ has completed by } t \},$$

the total true service of the higher-ranked jobs that have finished by t . Such a job arrived at or after a_q , so it necessarily ran while q waited. Job q is allowed a budget $\text{budget}(q, t)$

on that quantity, and the fired set at a dispatch epoch is $E(t) = \{q \text{ waiting} : \text{over}[q](t) \geq \text{budget}(q, t)\}$. At every dispatch epoch, that is, at every instant at which a server is free and the queue is non-empty, the wrapper serves the minimum-rank member of $E(t)$ when $E(t)$ is non-empty and otherwise serves whatever A chooses (Figure 2).

Serving the minimum-rank member matters when budgets differ between jobs. It guarantees that whenever a job j is dispatched ahead of a lower-ranked waiting job i , the job i was not in $E(t)$, hence $\text{over}[i](t) < \text{budget}(i, t)$ at that very instant. The per-job bound of Section 6 rests on that step. With a constant budget $\text{budget}(q, t) \equiv B$ the quantity $\text{over}[q](t)$ is non-increasing in rank, so $E(t)$ is non-empty exactly when the queue head belongs to it and the rule degenerates to a test on the head. Setting $B = 0$ recovers FCFS.

The cap carries the promise; the shape does not.

The guarantee of Section 6 reads the budget only through an upper bound on it. If $0 \leq \text{budget}(q, t) \leq B_{\max}$ for every waiting job and every instant, then every job waits at most $W_{\text{FCFS}} + B_{\max}/k + (3 - 2/k)L$, for every base policy, every arrival sequence and every set of predictions. Everything below the cap is a design freedom: it decides how much of the base policy's benefit survives and which jobs bear the harm, and it changes the promise not at all. We study the one-line family

$$\text{budget}(q, t) = \min(B_0 + \gamma n_q + \eta k(t - a_q), B_{\max}),$$

with $B_0, \gamma \geq 0$, $0 \leq \eta < 1$ and n_q the number of jobs waiting when q arrives, and read three shapes off it: *constant* ($\gamma = \eta = 0$), *age-relative* ($\eta > 0$), and *queue-length* ($\gamma > 0$). The coefficient η says how much of the system's capacity, over a job's own waiting time, may go to jobs that arrived after it. In other words, the allowance grows with the wait a job has already had. We chose a fraction of capacity rather than a number of seconds because it is dimensionless: it needs no retuning when k or the load changes. The term γn_q instead widens the allowance only for jobs that arrive into a queue that is already long, and it is flat in the wait. The quantities B_0 , γ and $B_{\max}$ are measured in work and scale with k .

Only the age-relative shape carries a second, multiplicative bound, and only when $\gamma = 0$: $W < (W_{\text{FCFS}} + B_0/k + (3 - 2/k)L)/(1 - \eta)$. With $\gamma > 0$ the constant part of the budget differs from job to job. The multiplicative form then holds per job, with that job's own constant, and the operator has no single number to state in advance (Supplementary Section S4.6). For $\gamma > 0$ we claim the additive bound alone.

Parameters from a service promise.

The operator states G , the number of seconds by which any job may exceed its FCFS waiting time, and reads off $B_{\max} = k(G - (3 - 2/k)L)$, the inversion of the uniform certificate proved in Section 6. No data are needed for this step. The rule issues no positive budget below $(3 - 2/k)L$; the zero budget remains admissible and reproduces FCFS exactly. Section 6 separately gives an L -scale floor, but only for this wrapper family when it must protect against every opaque base-policy proposal sequence. The promise G follows from $B_{\max}$ alone. The shape parameters B_0 , γ and η decide how much of the base policy's benefit survives and which jobs pay for it, so we choose them on a validation trace that excludes the test period. That pool is held out of the selection and not of the development process, and Section 8.2 states what the selection therefore does and does not establish. The rule is fixed before the grids are run. Supplementary Section S5 states it in full, and Supplementary Table S4 gives the grids and what they selected.

Algorithm 1 Overtake-budget guard wrapped around a base policy A

```

1: parameters: cap  $B_{\max} \geq 0$ ; shape  $B_0, \gamma \geq 0, \eta \in [0, 1]$ ; base policy  $A$ 
2: state: waiting set  $Q \leftarrow \emptyset$ ;  $\text{over}[q] \leftarrow 0$  for every job  $q$ 
3: while some job has not started do
4:    $t \leftarrow$  next event time
5:   for all jobs  $c$  completing at  $t$  do  $\triangleright C_c$  becomes known here and nowhere earlier
6:     for all  $q \in Q$  with  $q < c$  in rank do
7:        $\text{over}[q] \leftarrow \text{over}[q] + C_c$ 
8:     end for
9:   end for
10:  for all jobs  $i$  arriving at  $t$  do
11:     $n_i \leftarrow |Q|$ ;  $Q \leftarrow Q \cup \{i\}$ ;  $\text{over}[i] \leftarrow 0$   $\triangleright n_i$  fixes  $i$ 's constant part
12:  end for
13:  while a server is free and  $Q \neq \emptyset$  do  $\triangleright$  dispatch epoch
14:     $E \leftarrow \{q \in Q : \text{over}[q] \geq \min(B_0 + \gamma n_q + \eta k(t - a_q), B_{\max})\}$ 
15:    if  $E \neq \emptyset$  then
16:       $j \leftarrow$  the minimum-rank member of  $E$   $\triangleright$  guard fires
17:    else
18:       $j \leftarrow A(Q, t)$   $\triangleright$  base policy decides
19:    end if
20:    dispatch  $j$  on the free server;  $W[j] \leftarrow t - a_j$ ;  $Q \leftarrow Q \setminus \{j\}$ 
21:  end while
22: end while

```

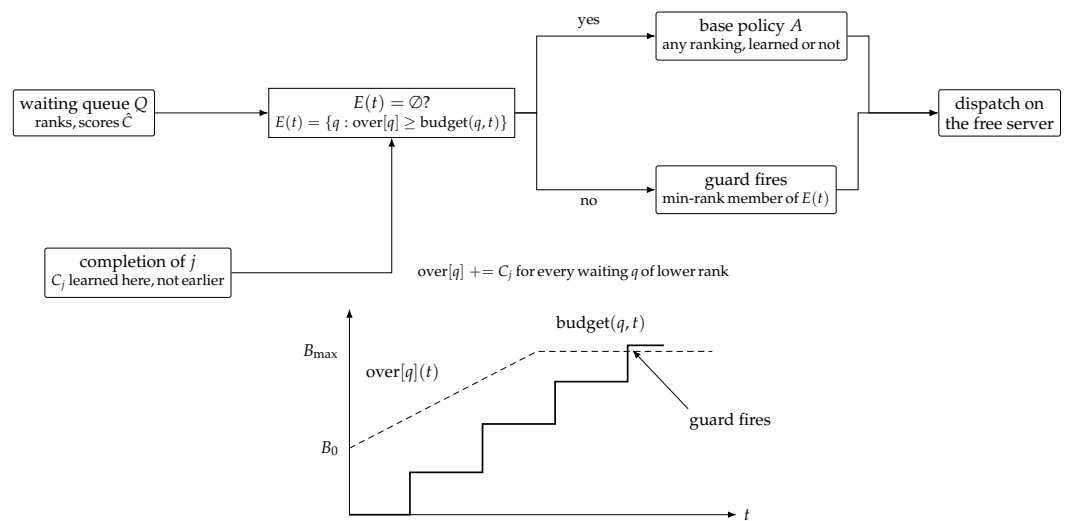


Figure 2. The guard of Algorithm 1 wrapped around a base policy (above) and the accounting it runs for one waiting job (below). Only completed work is charged, because C_j is unknown until j finishes, so $\text{over}[q]$ is a step function with a step at each completion of a higher-ranked job; at a dispatch decision, q is eligible if its account reaches its current budget. The guard dispatches the smallest-ranked eligible job, which need not be q . The budget drawn is the age-relative shape, rising at rate ηk from B_0 and capped at $B_{\max}$; the guarantee depends on the cap alone, so any curve under the dashed ceiling carries the same promise. The lower panel is schematic. Sources: the definitions in Section 5.2; $B_{\max} = k(G - (3 - 2/k)L)$ is Corollary 3.

What is charged, and why no service time is needed at dispatch.

The budget is charged the true executed work of overtakers, and only when they complete. At the instant the guard decides, it has read C_j only for jobs that have already finished, and a job still in service contributes nothing. That is what makes the wrapper implementable in a system that learns a job's cost at its completion, and it is also why the bound carries an additive term in place of an exact accounting. When the last overtaker of i is dispatched, up to $k - 1$ further overtakers may be in service with their work not yet charged, and the overtaker being dispatched is not charged either, so the work that actually passes i can exceed the budget by at most kL . Under the timeout at L , executed work means $\min(C_j, L)$; the log records that quantity and the guard charges it, so no assumption on raw sizes enters.

Cost per event.

Algorithm 1 is written for readability rather than for execution, since its inner loop over waiting jobs at each completion is linear. A Fenwick tree over completed work and a segment tree over the window of waiting ranks make each event cost $O(\log n)$. All budget arithmetic is carried in integer microseconds, so the firing test is exact and no floating-point clock enters the schedule. Supplementary Section S5 gives the two structures and the rewriting of the firing test. A promise in seconds thus becomes a number the dispatcher can check, and the dispatcher never reads a prediction to check it.

Why the budget is measured in work and not in positions.

Earlier bounded-overtaking mechanisms count passes [39,40] or limit service to the oldest jobs [42] instead. A count is loose against a work budget by the ratio $L/\bar{C}$, 150 on the main trace ($L = 60$ s against a mean cost of 0.40 s); Supplementary Sections S4.4 and S4.9 give the argument, and Section 8 measures what the looseness costs on a real load.

5.3. Compared Policies

Table 1 names the policies evaluated in Section 8 and fixes the names used in every table of this paper; Supplementary Section S5.7 states what each one is for at length. Guard(G) wrapped around SPJF-E is the method; the others separate its parts and test it.

Table 1. Policies compared in the experiments, and the name each carries throughout.

Name	Role
FCFS	Deployed baseline, and the reference the guarantee is stated against
SJF	Undeployable reference that sorts by true service time; the denominator of the gap closed
SPJF-E	Ranks by the expected-cost score of Section 5.1, unguarded
SPJF-log	The same pipeline on the log scale, and the other candidate scores of Section 5.1
Guard(G)	SPJF-E wrapped in the overtake-budget guard at the promise G . This is the method
Guard-fixed(G), Guard-age(G), Guard-queue(G)	The same wrapper with the shape the rule picks inside one grid. Ablation rows
Fixed(G)	The same wrapper at the equal-promise budget $\text{budget} \equiv B_{\max}$; infeasible under the harm rule
Skip(G)	Position-count guard, finite-skip family, counted at dispatch (Supplementary Section S4.4)
Corrupted scores	SPJF-E and Guard(G) under a reversed order, a random order and an inverted-tail score
Counterexamples	Retained traces from two audits and adversarial review, checked against the bounds

Every simulation that includes the guard asserts the per-job bound of Section 6 job by job inside the run (Section 8.6).

6. Theory

An operator who puts a prediction-driven scheduler in front of students wants to know the worst that can happen to one job, not what happens on average. Everything in this section is therefore stated for one fixed run: one arrival sequence, one vector of service times, one realisation of whatever randomness the base policy uses. We call such a statement pathwise. No stationarity, no independence and no assumption about score quality enters a proof, and the upper bounds hold even against a base policy that knows the future. The section has three parts. An identity ties a job's delay relative to first-come first-served to the work that overtook it; two bounds turn that identity into the guarantee of Algorithm 1; and a set of constructions shows that each constant in those bounds is an exact supremum, so that no smaller constant would do. Every statement kept here comes with the idea of its argument; Supplementary Sections S4 and S8 give every deferred proof, every extremal construction and the statements this section only summarises, S8 opening with the idea of each construction beside it. Sections 6.7 to 6.9 then add a clock rule, an absolute bound and late completion reports; of their constants only the late-report term is shown to be attained.

6.1. Conventions

We use the model of Section 3 throughout, and fix three readings the proofs depend on. The dispatches at one instant form a *phase*, ordered as the policy performs them, and the dispatch sequence behind $j \prec i$ is the concatenation of the phases; first-come first-served dispatches in increasing rank inside a phase. A job dispatched in the same phase as i executes no work during $[a_i, s_i^P)$, that interval ending at the instant of the phase, yet contributes its whole C_j to In_i . Remaining work at a_i is read *after* the arrivals at a_i , so a job of rank below i arriving at exactly a_i counts with its full service time.

6.2. The Workload Gap between Two Policies

Write $U_Q(t)$ for the unfinished work at time t under policy Q , counting every job with $a_j \leq t$, and $N_Q(t)$ for the number of jobs present. The unfinished-work vector of a k -server queue obeys the Kiefer–Wolfowitz recursion [73], and first-come first-served minimises that vector in the weak-majorisation order for systems in which arrivals are allocated to servers in a manner that does not depend on their service times [69]. The policies compared below are not restricted that way, so what carries the single-server argument to $k > 1$ here is a two-sided bound between two arbitrary policies rather than an ordering.

Lemma 1 (Workload comparison). *Let Q_1 and Q_2 be non-preemptive work-conserving k -server policies run on the same input. Then $|U_{Q_1}(t) - U_{Q_2}(t)| \leq (k - 1)L$ for every t . For $k \geq 2$ the inequality is strict, $|U_{Q_1}(t) - U_{Q_2}(t)| < (k - 1)L$, and the constant is the exact supremum, approached but never attained. At $k = 1$ both sides vanish: one server executes work at rate 1 whenever at least one job is present, so every work-conserving policy has the same unfinished work at every instant.*

Proof idea. The difference $D = U_{Q_1} - U_{Q_2}$ is continuous and piecewise linear. At every instant at which U_{Q_1} exceeds $(k - 1)L$ it cannot increase, since more than $k - 1$ jobs are then present under Q_1 and every server of Q_1 is busy. Tracking D back to the last instant at which U_{Q_1} was below the threshold gives the bound. The proof, the strictness, and the cascade that shows the constant cannot be lowered are in Supplementary Sections S4.1 and S8.1. $\square$

Two work-conserving policies can therefore drift apart in unfinished work, but only by what $k - 1$ servers can hold; at one server they never drift at all.

6.3. The Net-Overtake Identity

For a job i and a policy Q , let R_i^Q be the remaining work at time a_i , under Q , of the jobs of rank below i , read after the arrivals at a_i ; and let ρ_i^Q be the remaining work at s_i^Q of the jobs dispatched strictly before i in Q 's dispatch sequence that are still in service then. Write

$$\Delta_i = k(W_P[i] - W_{\text{FCFS}}[i]) - (\text{In}_i - \text{Out}_i).$$

Theorem 1 (Net-overtake identity). *For every non-preemptive work-conserving k -server policy P , every input and every job i ,*

$$\Delta_i = (R_i^P - R_i^{\text{FCFS}}) - \rho_i^P + \rho_i^{\text{FCFS}}, \quad |\Delta_i| \leq 2(k-1)L,$$

with strict inequality for $k \geq 2$; equivalently $|W_P[i] - W_{\text{FCFS}}[i] - (\text{In}_i - \text{Out}_i)/k| \leq (2 - 2/k)L$. At $k = 1$ the error term vanishes and $W_P[i] - W_{\text{FCFS}}[i] = \text{In}_i - \text{Out}_i$ exactly.

Proof idea. Job i waits throughout $[a_i, s_i^P]$, so every server is busy there and exactly $k W_P[i]$ units of work are executed, none of it on i . Splitting that work by rank gives the busy-period accounting

$$k W_P[i] = R_i^P + \text{In}_i - \text{Out}_i - \rho_i^P, \quad \rho_i^P := \rho^{\text{old}} + \rho^{\text{new}}, \quad (1)$$

where ρ^{old} and ρ^{new} are the work still unfinished at s_i^P of, respectively, the jobs dispatched strictly before i and the overtakers, and a same-phase overtaker contributes C_j to both In_i and ρ^{new} and nothing to the executed work. The jobs counted in ρ_i^P occupy distinct servers and none of them occupies the server that takes i , so $0 \leq \rho_i^P \leq (k-1)L$. Under first-come first-served $\text{In}_i = \text{Out}_i = 0$, and Lemma 1 applied at a_i bounds $|R_i^P - R_i^{\text{FCFS}}|$ by $(k-1)L$, strictly for $k \geq 2$; subtracting the two instances of (1) gives the identity. Each direction uses only two of the three ranges, which is why the slack is $2(k-1)L$ and not $3(k-1)L$, and at $k = 1$ all three terms vanish. Supplementary Section S4.3 gives the proof in full. $\square$

Read from right to left, the identity turns a bound on overtaken work into a bound on delay, with nothing else assumed about the policy.

Corollary 1 (A work budget is sufficient). *If a policy keeps $\text{In}_i - \text{Out}_i \leq Z$ for every job, in particular if it keeps $\text{In}_i \leq Z$, then $\text{excess}_P[i] \leq Z/k + (2 - 2/k)L$ for every job.*

A scheduler that never lets more than Z units of work pass a waiting job therefore holds every wait within $Z/k + (2 - 2/k)L$ of arrival order, whatever it ranks by. That is the whole mechanism of Algorithm 1.

6.4. The Constant $2(k-1)L$ Is Exact

The error term of Theorem 1 is not an artefact of the proof, although neither search nor real input reaches it: the extremal instances are geometric cascades needing hundreds of jobs, which nothing in a replayed workload arranges. Section 8.6 and Supplementary Section S3.13 report how far below the constant the measured residual stays, on small instances and on the trace alike.

Theorem 2 (Tightness of Theorem 1). *For every fixed $k \geq 2$, service bound $L > 0$, and $\varepsilon > 0$ there are inputs, work-conserving policies P and jobs i with $\Delta_i > 2(k-1)L - \varepsilon$, and there are*

others with $\Delta_i < -2(k-1)L + \varepsilon$. Hence $\sup |\Delta_i| = 2(k-1)L$, a supremum that is never attained.

Proof idea. Both witnesses start from the cascade of Supplementary Section S8.1, which drives $U_P - U_{\text{FCFS}}$ to $(k-1)L(1 - ((k-1)/k)^m)$ in m rounds. The upward witness then injects $k-1$ long jobs, the victim, and a stream of unit jobs the base policy prefers, so that first-come first-served starts the victim at once with $\rho_i^{\text{FCFS}} = (k-1)L$ while P holds it until the stream runs out; the downward witness is the mirror image, with P dispatching $k-1$ jobs of service L in the victim's own phase. The construction is first given at scale $\hat{L} = k^m$, and every arrival time and service time is then multiplied by $L/\hat{L}$, so the limiting sequence keeps the stated L fixed. Supplementary Section S8.2 gives both witnesses in full, with the exact values they attain. Non-attainment follows from Lemma 1. $\square$

The two directions do not have the same content. Upward, the extremal value is a statement about delay. Downward, half of it is the bookkeeping convention that charges an overtaker dispatched in the victim's own phase, which executes nothing while the victim waits. Supplementary Section S4.3 gives the remark in full, with the same-phase shares, the executed-work variant under which the upward supremum is unchanged and the best downward witness falls to $(k-1)L$, and the two reasons we keep the dispatch-sequence convention.

6.5. Necessity, and What a Count of Overtakes Costs

Theorem 1 has a converse. A policy that promises anything about individual jobs is forced to bound overtaken work.

Lemma 2 (Outbound work under a promise). *Let i be a job, and suppose $\text{excess}_P[i] \leq G$ and $\text{excess}_P[j] \leq G$ for every job j counted in Out_i . Then $\text{Out}_i \leq k(G - \text{excess}_P[i] + L)$.*

Proof idea. Every job counted in Out_i starts inside a window of length $G - \text{excess}_P[i]$, and of the jobs one server starts inside such a window all but the last complete inside it. Supplementary Section S4.7 gives the proof. $\square$

A promise therefore limits not only the work that may pass a job but the work that job may itself pass. That second limit is what makes the converse work.

Corollary 2 (A work budget is necessary). *If P satisfies $\text{excess}_P[j] \leq G$ for every job on some input, then $\text{In}_i \leq kG + (3k-2)L$ for every job i .*

Proof idea. Add Lemma 2 to Theorem 1, where the terms in $\text{excess}_P[i]$ cancel. Supplementary Section S4.7 gives both proofs. $\square$

Together the two corollaries say that bounding In_i and offering a per-job guarantee are the same requirement, up to an additive $O(kL)$ in the budget. Neither says that Algorithm 1 is the least restrictive rule with its guarantee, and we do not claim it.

Bounded-overtaking mechanisms in the literature limit instead how many times a job may be passed [39,40], or how far down the queue service may reach [42]. Inside the model of Section 3 such a count is not necessary for a per-job guarantee, and it gives one only at the worst-case charge of L per pass. That charge makes it loose against a work budget by the ratio $L/\bar{C}$, 150 on the main trace ($L = 60$ s against a mean cost of 0.40 s). Supplementary Section S4.4 states this as a remark and proves both halves, and it carries the conversion from a count to a promise that the Skip baseline of Section 5 uses.

6.6. The Guarantee of the Guard

We now bound what Algorithm 1 does. Nothing below restricts the base policy, which may be randomised, learned, or clairvoyant. Nothing restricts the budget rule either, beyond the cap: it may depend on the job, may decrease over time, and may take different values at two dispatches made at the same instant.

Theorem 3 (Guard bound). *Fix a base policy A , a cap $B_{\max} \geq 0$, and a budget rule that respects the cap, that is, a function $\text{budget}(q, t)$ satisfying $0 \leq \text{budget}(q, t) \leq B_{\max}$ for every waiting job q and every dispatch epoch t ; run Algorithm 1 with that rule. For every input and every job i , writing $W = W_P[i]$ for the wait under the wrapper,*

$$\text{In}_i < B_{\max} + kL, \quad (2)$$

$$W < W_{\text{FCFS}}[i] + B_{\max}/k + (3 - 2/k)L, \quad (3)$$

where (2) is asserted whenever i has at least one overtaker.

Proof idea. Read the last overtaker of i : at the instant it is dispatched, i is waiting and is not in the fired set, so $\text{over}[i]$ is below its budget there. The overtakers dispatched earlier and not yet finished occupy at most $k - 1$ distinct servers, and the last one adds a further L , which is the kL of (2). Theorem 1 with $\text{Out}_i \geq 0$ turns that into (3). Supplementary Section S4.5 gives the proof in full. $\square$

Nothing in the proof reads the budget rule except through the cap, so the shape of the budget below the cap is a free choice and the promise is unaffected by it. One shape carries a second bound, in which the allowance grows with the wait the job has already had rather than being flat in it. For the age-relative budget $\text{budget}(q, t) = \min(B_0 + \eta k(t - a_q), B_{\max})$, with one constant $B_0 \geq 0$ for every job and $\eta \in [0, 1)$, the wait obeys $(1 - \eta)W < W_{\text{FCFS}}[i] + B_0/k + (3 - 2/k)L$; Supplementary Section S4.5 states that as a proposition and proves it. The same bound written with a common B_0 is false for the queue-length shape of Section 5, by a margin that grows with γ , so for $\gamma > 0$ we claim Theorem 3 alone and Supplementary Section S4.6 gives the counterexample. The cap, and with it the promise, is unaffected either way.

A scheduler that learns C_j at completion cannot charge work in progress, so only completed work is charged; the kL in (2) comes from that, and it is what forces the additive term. Inverting (3) gives the parameter rule of Section 5:

Corollary 3 (The cap that offers a promise). *Setting $B_{\max} = k(G - (3 - 2/k)L)$ makes Algorithm 1 offer the service guarantee G , whatever the budget shape below the cap and whatever the base policy A . The choice is available when $G \geq (3 - 2/k)L$, and gives a positive budget only when the inequality is strict; at equality $B_{\max} = 0$ and the wrapper is first-come first-served.*

This is the line an operator uses: a promise in seconds goes in, a budget in units of work comes out, and nothing about the base policy has to be known.

One assumption can be traded for a smaller constant. If the service enforces a per-job run-time cap $\ell_j \leq L$ known at arrival, charging it at dispatch and refunding it at completion keeps In_i within the budget, so the kL of (2) goes and the additive constant falls from $(3 - 2/k)L$ to $(2 - 2/k)L$, a drop of exactly L , the promise at $k = 1$ becoming excess $\leq B$ exactly. The information model is then not the completion-only one of Section 5 and the mechanism is conservative backfilling's [32,33], so Supplementary Section S7.6 carries it, with the single-server maximality and the counterexample that rules maximality out for $k \geq 2$.

Neither constant in Theorem 3 can be improved. Under a constant budget B the term B/k is reached exactly, on a queue of k jobs of service L followed by a stream of unit jobs. The additive constant $(3 - 2/k)L$ of (3) is the exact supremum at every k , approached by a cascade that has to be run for enough rounds. Both sources of slack, the constant of Theorem 1 and the kL of (2), are saturated on that one instance, where the excess is real waiting time rather than bookkeeping. Supplementary Section S8.3 states this as a proposition, gives both constructions, and reports what a hill-climb over small instances suggests instead, beside the size the witness actually needs.

A guard that never fires leaves the base policy alone, and the condition is checkable on the base policy's own run, before the wrapper is placed around it. If $\text{over}[q](t)$ stays below $\text{budget}(q, t)$ for every waiting job at every dispatch epoch of that run, the wrapped run is that run, with the same dispatch sequence and the same waits. Supplementary Section S4.2 states this as a proposition, with the two conditions under which a constant budget meets its hypothesis and the two counterexamples that make consistency one-sided.

6.7. A Clock Keeps the Same Promise

Deployed schedulers guard against starvation with a maximum waiting time. Recent inference servers promote a request once its wait passes a fixed threshold [56] or a multiple of its service time [57], and state no bound on the wait. In the simplest form, once the oldest waiting job has waited θ it is served next, and until then the base policy decides. Call this $\text{Timeout}(\theta)$, and read "oldest" as the waiting job of minimum rank, so that ties in arrival time are broken by input order and not by the base policy. The rule reads arrival times and a clock, and never a service time. It carries the same kind of promise as the guard.

Proposition 1 (Timeout bound). *Fix a base policy and $\theta \geq 0$. For every input and every job i with at least one overtaker, $\text{Timeout}(\theta)$ gives $\text{In}_i < k(\theta + L)$, and every job waits*

$$W < W_{\text{FCFS}}[i] + \theta + (3 - 2/k)L.$$

Proof idea. From $a_i + \theta$ on, the minimum-rank waiting job has waited at least θ , so every dispatch until s_i serves a job of rank below i . Each overtaker of i therefore starts before $a_i + \theta$ and ends no later than L after its start. The overtakers that share a server run one after another inside $[a_i, a_i + \theta + L)$, and summing over the k servers bounds In_i . Theorem 1 with $\text{Out}_i \geq 0$ turns that into the bound on the wait. Supplementary Section S4.11 gives the proof in full. $\square$

At $\theta = G - (3 - 2/k)L$ the clock makes the promise that the guard makes at $B_{\max} = k\theta$. The two rules differ in what they charge a waiting job for. The clock charges it for all the work the servers do while it waits, $k(t - a_q)$. The guard charges it only for the completed work of jobs that arrived after it. The difference is mostly the work of older jobs, which first-come first-served would have done anyway, plus the work of overtakers still in service. Near saturation that work is large. A job that arrives behind a backlog is charged for the backlog, the clock fires for most of the queue, and the schedule falls back to arrival order at the loads where ordering by cost pays most. Section 8.5 measures the difference.

The two rules also combine. Serve the minimum-rank waiting job if it has waited θ , and otherwise run Algorithm 1. When a job is served ahead of a waiting job i of lower rank, i is then outside the guard's fired set, which is all Theorem 3 uses; and every overtaker of i still starts before $a_i + \theta$, which is all Proposition 1 uses. Both bounds hold at once. The order matters: a rule that serves the guard's fired job first can pass over an aged head

when budgets differ between jobs, and it breaks the clock's bound for the queue-length and age-relative shapes (Supplementary Section S4.11 gives the instances). The clock's half reads no completion report, so the combined rule keeps $W < W_{\text{FCFS}} + \theta + (3 - 2/k)L$ when reports arrive late or not at all.

6.8. An Absolute Bound from the Arrival Log

A promise relative to first-come first-served is a margin over a queue that nobody runs. The arrival log alone gives an absolute one, which an operator can state as a number of seconds. Let V_i^- be the work that a single server of rate k , fed the same jobs, would still hold at a_i from the jobs of rank below i : $V_1^- = 0$ and $V_i^- = \max(0, V_{i-1}^- + C_{i-1} - k(a_i - a_{i-1}))$. The largest value $\sigma = \max_i(V_i^- + C_i)$ is the depth of the smallest token bucket of rate k that the arrivals, counted in work, conform to, which is the burst parameter of network calculus [74,75].

Proposition 2 (Absolute bound). *For every non-preemptive work-conserving policy and every job i , $k W[i] \leq V_i^- + (k - 1)L + \text{In}_i$. Hence $W_{\text{FCFS}}[i] \leq (V_i^- + (k - 1)L)/k$, and under Algorithm 1 with cap B_{max}*

$$W < \frac{V_i^- + B_{\text{max}}}{k} + \left(2 - \frac{1}{k}\right)L \leq \frac{\sigma + B_{\text{max}}}{k} + \left(2 - \frac{1}{k}\right)L.$$

Proof idea. The accounting (1) gives $k W[i] \leq R_i + \text{In}_i$. A work-conserving k -server queue never holds more than $(k - 1)L$ above the fluid server. Whenever it does, all k servers are busy, since otherwise the queue is empty and at most $k - 1$ jobs in service hold at most $(k - 1)L$, and then it drains at the fluid rate. So $R_i \leq V_i^- + (k - 1)L$, and Theorem 3 bounds In_i . Supplementary Section S4.11 gives the proof in full. $\square$

The comparison with the fluid server is the argument Groszof, Scully and Harchol-Balter use for multiserver shortest-remaining-processing-time, where it bounds by kx the difference between the k -server system and the fast single server in the work of jobs below a size cutoff x [76]. Here it covers every non-preemptive work-conserving policy and all the work, and the constant is $(k - 1)L$.

At $k = 1$ the fluid server is the queue, $V_i^- = W_{\text{FCFS}}[i]$, and the proposition is Theorem 3 again. For $k \geq 2$ it is sharper than a bound that goes through W_{FCFS} , which would pay the $2(k - 1)L$ of Theorem 1 a second time. An operator who promises that no job waits longer than D , over a period whose arrivals conform to a bucket of depth σ , sets $B_{\text{max}} = kD - \sigma - (2k - 1)L$: the capacity over the deadline, less the burst, less $2k - 1$ job-limits. The clock obeys the same bound with B_{max} replaced by $k\theta$. The comparison with a single fast server has a history of false sample-path claims. Wolff [77] showed that previously derived sample-path upper bounds on the work in a multichannel queue are false. The statement here also holds on every sample path. What separates it from the refuted bounds is the additive $(k - 1)L$, which the service cap L makes finite, and it is proved directly. Section 8.6 measures how close the bound comes on the traces, and how well one period's σ predicts the next.

6.9. Completion Reports That Arrive Late

Theorem 3 assumes that a completion is charged before the next dispatch. In a distributed service the workers report completions over a network, and a server may pull its next job before its report has reached the counter. Suppose every charge reaches the counter within δ of the completion, while servers still pull the instant they free.

Proposition 3 (Late charges). *Under Algorithm 1 with a shared counter and charges delayed by at most δ , every job i with an overtaker has $\text{In}_i < B_{\max} + kL + k(L + \delta)$, and*

$$W < W_{\text{FCFS}}[i] + B_{\max}/k + (4 - 2/k)L + \delta.$$

The extra $L + \delta$ is attained in the limit at $k = 1$, and approached to within 0.2% at $k = 2$ on the constructed instances. If each server applies its own completion report in the same step as its next pull, and only other servers' reports travel late, Theorem 3 holds unchanged for every δ , lost reports included.

Proof idea. At the last overtaker's dispatch the charged part is below the budget, as before. The uncharged overtakers are those in service, as in Theorem 3, and those that finished in the last δ and whose charge has not arrived. On one server the latter are at most one job that started before that window and jobs that fit inside it, so they hold less than $L + \delta$. When a server reports before it pulls, any stale completion must belong to a server that finished at the dispatch instant itself and has not yet pulled; that server holds no job in service, so the per-server uncharged work is at most one job, as in Theorem 3. Supplementary Section S4.12 gives both proofs and the instances that attain the extra term. $\square$

At $k = 1$ any $\delta > 0$ already costs a whole L : the single server pulls its next job at the instant it finishes one, before that job's charge can arrive. Pulling ahead of the report therefore changes the promise, and reporting on the pull removes the change. The combined rule of Section 6.7 gives a second remedy that needs no protocol at all, since its clock half keeps $W < W_{\text{FCFS}} + \theta + (3 - 2/k)L$ for any delay.

6.10. Which Assumptions Can Be Dropped

Four of the standing assumptions can go, and one cannot. The global service bound L may be replaced by the prefix maximum of the service times seen so far, provided it is read at the right instant, and the right instant is not the same for every statement. Servers may run at different speeds, the budget then being divided by total capacity rather than by the server count. The per-job time limit is already inside the model, because C_j counts executed work. A fixed setup paid before every service is absorbed into the job. Work conservation is the assumption that cannot go: if a server may idle while jobs wait, the identity fails by exactly the forced idle time, and release constraints are the same phenomenon. Supplementary Section S4.10 states all five as two propositions, with the sharp forms and the counterexample that fixes where the prefix maximum is read.

6.11. A Floor of Order L , and What a Guarantee Costs

No choice of budget removes the last L of the additive constant. A job can be displaced before anything at all has completed, and at that instant a wrapper that charges completed work has nothing to charge.

Proposition 4 (Impossibility). *Fix a budget rule that assigns a positive budget to every waiting job at the first dispatch epoch. Then there are a base policy A that observes a job's service time only at its completion, and an input, such that Algorithm 1 run around A with that rule leaves some job with excess $\geq L$. Consequently no promise $G < L$ can be made by a wrapper of this family that holds for every base policy, or for one prediction-ranked base policy under arbitrary predictions, which is the generality in which Theorem 3 is stated.*

Proof idea. Put $k + 1$ jobs of service L into an empty system at $t = 0$: nothing has completed, so no waiting job fires, the wrapper accepts the base policy's choices, and the first choice beyond the first k ranks pushes back a job that first-come first-served starts at once,

leaving it with excess L . For a base policy that ranks by predictions, such as SPJF-E, the predictions in the input already force this and A need not be chosen. The hypothesis can be neither dropped nor tightened: as a claim about policies the statement is false, since first-come first-served itself has excess 0 on every input, and a rule that is 0 at the first dispatch epoch and positive later falls to the same instance placed behind a saturating prefix. Supplementary Section S4.8 gives the proof and the case analysis that makes those cases exhaustive for the budget of Algorithm 1, which either makes the wrapper first-come first-served around every base policy or admits a base policy and an input with excess at least L . $\square$

At one server the floor is the whole story. With a constant budget $B > 0$ the bound $B + L$ of Theorem 3 is the exact supremum of the excess, approached and not attained; at $B = 0$ the wrapper is first-come first-served and the excess is exactly zero. Supplementary Section S8.4 gives the single-server family that approaches $B + L$ without reaching it.

The guarantee also caps how much reordering can gain at all, and not only how much it may cost a single job.

Theorem 4 (Price of the guarantee). *Take m jobs of service L and n jobs of service $\sigma < L$, the long ones ranked first, all arriving at $t = 0$, and let P be a non-preemptive work-conserving policy with $\text{excess}_P[j] \leq G$ for every job. Define $D_0 = mn(L - \sigma)/k$, $E = 2(1 - 1/k)L(m + n)$ and $A = (kG + (3k - 2)L)/(n\sigma)$. If $D_0 > E$, equivalently $mn(L - \sigma) > 2(k - 1)L(m + n)$, then*

$$\frac{\bar{W}_{\text{FCFS}} - \bar{W}_P}{\bar{W}_{\text{FCFS}} - \bar{W}_{\text{SJF}}} \leq A + \varepsilon_+, \quad \varepsilon_+ = \frac{E(1 + A)}{D_0 - E}.$$

For fixed k, L, σ, G , the finite-sample term is $O((m + n)/(mn))$.

The proof is in Supplementary Section S8.5, which counts inverted pairs in two ways and so reads the pairwise-interchange argument behind Smith's rule [78] on an arbitrary arrival sequence. The right-hand side is capacity times guarantee, divided by the work that wants to overtake; it is linear in G and saturates at 1 when kG reaches the total service of the impatient class. Because it applies to every policy with the guarantee and not only to Algorithm 1, it describes the trade-off an operator faces rather than a weakness of this wrapper. The additive $(3k - 2)L$ in it is slack: it comes from applying Corollary 2 to each long job separately, where the long jobs can be charged together, and at $k = 1$ the aggregate argument in the same place removes it. There the achievable fraction is known exactly: it is $\lfloor G/\sigma \rfloor / n$, capped at 1, which Supplementary Section S8.5 states as its own proposition and checks exhaustively over 1161 parameter settings.

6.12. Achievability and Maximal Permission

Two further results say how far the guarantee can be pushed, so that the earlier ones are not read only as a robust wrapper beside a necessary budget. On the two-size batch family of Theorem 4, the guard around exact shortest-job-first closes a fraction of the first-come-first-served-to-SJF gap that meets the upper bound of that theorem, up to $2(3k - 2)L/(n\sigma)$ and two finite-sample terms. At one server, with integer sizes and budgets, the guard that fires at over $\geq G - L + 1$ permits every action a wrapper with the promise G could safely permit, in a class that sees ranks, completions, revealed sizes and the base policy's proposals but never an unfinished size. Neither result gives instance optimality on arbitrary inputs, and for $k \geq 2$ the exact optimum stays open. Supplementary Section S8.6 states both, with the definition of the wrapper class, the single-server frontier they imply and the proofs; supplementary theory material records an unbounded instance-optimality counterexample and the limits of instance-dependent refinements.

Every uniform constant above is a multiple of L , apart from the report delay δ of Proposition 3 and the arrival burst σ of Proposition 2. Proposition 4 establishes such a floor for the wrapper family under arbitrary opaque base proposals, not for all scheduling mechanisms. Empirically, usefulness depends on size dispersion, score predictability and the arrival structure; the CI pools are negative cases rather than evidence of a necessary distributional condition. Section 8.8 quantifies that boundary on the evaluated workloads.

7. Traces and Workload Characterisation

7.1. The Traces

The two judging platforms record how long each submission ran; neither records what it waited for. The pool, its capacity and the queue are therefore ours, and this section says how we built them and what supports the construction. Two programming-assignment judging platforms supply the demand traces, and four further workloads bound the claim from outside that setting. What each trace can be used for is decided by whether its arrival times are observed or reconstructed, whether the queue wait is recorded, and how long a job may run there (Table 2).

Table 2. The traces, their role and what they record. “Arrivals” says whether the arrival instants are recorded, reconstructed proxies, or drawn; “Waits” whether the queue wait of each job is itself recorded.

Trace	Role	Size	Time fields	Arrivals	Waits
CodeBench [16]	main demand trace	4,611,325 blocks, 863,149 graded submissions in the development pool	one stamp per block, the end of the run	start proxy	no
ACcoding [17]	second judging platform	4,046,652 rows, 3,237,322 non-sealed	none in the release; submission id only	no, drawn	no
Serverless invocations [7]	method applies outside education	1,980,951 invocations over 14.000 d	modified end stamp and duration	start proxy	no
CI pool A [20,21]	simulator validation, light tail	191,970 runs over 21 d	created, scheduled, started, resolved	yes	yes
CI pool B	second validation pool	30,702 runs over 6.99 d	as above	yes	yes
Compute farm [18,19]	non-applicable case	9,054,066 jobs over 30.00 d, 8,441,207 serial with positive run time	submit, wait, run	yes	yes
OULAD [79]	calendar-driven load elsewhere	9 course-presentations, daily counts	day	—	—

Both CI pools and the compute farm carry a recorded queue wait. We validate the simulator against the CI pools, whose capacity is a countable set of physical machines that changes only on reimaging or quarantine. The released tables of the second judging platform have no timestamp column, only a submission id, so job order inside a group and job sizes are real while every arrival instant is drawn. The correlation between a group’s first id and its recorded start time is 0.9995 over 469 groups, but in 207 of 468 consecutive transitions the id ranges overlap, so the id is an ordering and not a clock. Its results are a check that the ordering mechanism transfers, not a wait in seconds.

7.2. What the Fields of the Main Trace Mean

Several fields of that trace had to be interpreted before a queueing model could use them. Supplementary Section S2 gives the audit that establishes the readings below, together with the comparison against the release statistics and the two truncated archives.

The timestamp on an execution block is the moment the result was written, not the moment the code was received. We therefore take done_j to be the recorded stamp and reconstruct the arrival as $a_i = t_i - C_i$, and we carry the opposite reading through the pipeline as a sensitivity with its own features and models. Stamps are recorded to the

whole second while costs are recorded to the microsecond, and that opens a leak. We close it by jittering each stamp deterministically. The leak is larger than we expected: a classifier given only the fractional part of the gap to the previous record separates heavy from light jobs with AUROC 0.8091 on the whole-second clock. After the jitter it scores 0.4499, which is chance. The forward models of the two clocks stay within 0.008 of AUROC of each other, so the repair costs nothing.

The observed regime has an upper ceiling of 60 s; this is not evidence of a documented enforced timeout. The last run to reach a longer time ends at 2019-08-28. The periods before that date stay out of the primary pool, and where used at all they carry the later limit, $C^{\text{cap}} = \min(C, 60)$ (Supplementary Table S3). Groups of runs by the same user that end in the same second are 7.53% of blocks and 26.28% of capped work. They overlap and stretch each other, so we neither delete them silently nor treat each as an independent arrival. We rebuild the scheduling trace twice, once with each group collapsed into one job and once with the groups removed (Supplementary Table S15).

7.3. What the Source Platform Identifies

The platform that produced the main trace creates a separate container for each student and runs their code inside it [80]. Its logs do not identify the waiting times of a shared judging queue. We use recorded durations and reconstructed start-time proxies as inputs to an assumed shared pool; its capacity, queue and service order are our design. Neither this construction nor the absence of shared-queue fields establishes that all source-platform work experienced no waiting.

Three things support the design. The first is that a fixed pool of judging hosts on one queue is a standard deployment. A widely used contest judging system recommends “one judgehost per 20 teams” [2] on dedicated machines, and the autograder study of Section 1 runs a configurable number of concurrent graders and measures the queue wait [1]. Dedicated per-entity capacity is also known to sit idle in workloads like this one, which two measurements outside our own traces record [81,82]. The second support is a measurement on the same demand. Replayed under the dedicated design, that demand keeps 1,636 containers alive at the 99th percentile, each executing code for 0.12% of the time it is alive, and a shared pool holds the 99th percentile of deadline-window waits under 1 s with 14 servers. The third is that the simulator reproduces waits somebody else recorded: refitting an effective capacity and a setup term on one chronological part of each continuous-integration window returns the held-out mean wait, the per-hour profile and the per-job ordering. Supplementary Section S2.3 gives all three in full. It carries what the two external measurements do not license, the six session reconstructions, the five chronological splits, the sharpness check, the one quantile that does not survive and the circularity of the original whole-window fit. The pooled side is simulated, so none of this supports a claim that the platform ever queued.

Capacity and physical checks.

The supplementary capacity sweep retains the complete offered stream and varies the number of servers; it uses the archived embedded Tweedie fit without refitting. The physical experiment executes FCFS, SPJF-E and a guard on the complete selected development window, with timed slot occupancy and recorded enqueue, dispatch and completion events. It tests an executing rule but does not transport submitted programs to new hardware. The selected empty-start window is distinct from a full-trace replay; its guarded p99 is worse and its job-level ideal-simulator agreement fails (Section 8.9, Supplementary Section S6).

7.4. Building the Shared Pool

A single class-term does not load one server, so we overlay several. The pool takes only the development terms in the observed 60 s regime, 40 class-terms in all, and replicates them 44 times, each copy aligned by weekday and hour and shifted by a whole number of weeks drawn per copy. Overlaying whole terms rather than compressing inter-arrival times preserves the daily and weekly cycle and each job's position relative to its own deadline. The result is 17,634,760 jobs over 30 weeks and 208.8 days. The three load levels share that trace and differ only in the pool size. Four of the five overlays take $k = 8/5/4$ and the fifth takes $7/5/4$, its work per busy hour being the lowest of the five; on the first overlay that gives busy-hour utilisations of 0.529, 0.847 and 1.059, and 0.502, 0.783 and 0.979 averaged over the five. A deadline window is the 24 hours before an assessment's own end, and 4,219,556 jobs arrive inside one. Five overlays are week-shifts of the same terms, so they are one construction and not five platforms, and our intervals resample whole weeks, which covers time and leaves the choice of terms not covered. A trace built from the last development term alone is not available this way, since it would need 190 copies of its 8 class-terms to load a pool of four.

7.5. What the Load Looks Like

Arrivals follow the course calendar, and the calendar predicts them. Under grouped-class cross-validation, predicting the number of executions per class 168 hours ahead gives log-scale RMSE 1.243 with lag features alone, 1.159 with calendar features alone, and 1.112 with both. A paired bootstrap over class-week blocks gives a combined-minus-lag-only difference of -0.1307 $[-0.1395, -0.1213]$. The supporting trace shows the same shape elsewhere. Across its nine course-presentations, the three days up to an assessment due date carry 1.29 to 2.95 times the clicks of the other test days, and a seven-day-ahead peak warning on distinct active students goes from F1 0.000 under a moving-average baseline to 0.593 once calendar features are added.

Costs are concentrated in a small fraction of jobs on every workload except the CI pools, which motivates the empirical cost-concentration screen (Supplementary Table S3, which also records where the analysis bound L comes from on every trace). Supplementary Section S2 gives what the arrival and service fields of each trace are, which of them are proxies, and how many jobs each analysis truncation touches. The traces give a demand model and a replayed queue, and no historical measurement of either.

7.6. Sealed Data

Three of the traces carry a part that was sealed before the method was designed. On the main trace it is three terms, 2023-1, 2023-2 and 2024-1; on the second judging platform it is the last 20% of the submission-id range, 809,330 rows, of which only the row count and the first id have been read; on the supporting trace it is the 2014 academic year. We will open only the main trace's sealed terms for this paper, once, after freezing the method, its parameters and the primary metric. The frozen pipeline covers those three terms and nothing else; the sealed parts of the other two traces stay shut, and we report on neither. The serverless trace, the compute farm and the two continuous-integration pools have no sealed part, so every figure we give for them is development data. No cost distribution, prediction or scheduling result has been computed on any sealed part, and every access to one is entered in a record in the repository. Every number in this section and in Sections 4 and 5 is a development number. The sealed evaluation remains pending in this manuscript; Section 8.2 states the current output scope.

8. Experiments

Ordering by the predicted number closes most of the gap to true shortest-job-first, and it leaves one job almost a hundred job-limits behind. What it costs to remove that job is what this section measures. The predictors are taken as given from Section 4. We ask what a scheduler should sort by (Section 8.4), what the per-job guarantee costs and what it gains (Section 8.5), whether the bounds of Section 6 hold on real traces (Section 8.6), and where the method applies (Sections 8.8 and 8.10).

8.1. Setup

The main scheduling comparisons replay the overlaid CodeBench pool built in Section 7.4. The three load levels differ only in the pool size and the five overlays only in the per-copy week shift, and every policy sees the same arrival instants and the same service times, so the comparison is pathwise. Supplementary Table S4 collects the configuration and the selection protocol; the metrics are defined in Section 3 and the policy names in Table 1, extended by the aging comparator below. Section 8.3 gives the headline under the exact protocol, in which every score uses only results available in the replay. The original-score ranking, budget-shape, corrupted-score, residual and single-server results retain the source result clock and are optimistic references, not estimates under the exact protocol. The capacity and physical experiments use an archived development fit and are reported separately. Table 4 and Supplementary Tables S6 and S16 report the plain percentage reduction against FCFS beside the fraction of the FCFS-to-SJF gap closed, because they are different numbers and the second is easy to read as the first.

8.2. What Was Selected on What

Three statements precede the numbers. The guard family and a coarse grid over (B_0, η) were explored on the evaluation trace before any validation-only selection existed, so the selection chooses inside a family already narrowed on the trace this section reports. Note that nothing here is held out. The validation pool is the development terms minus the last one. That last term is held out of the selection, not of the development process, and it cannot load a pool on its own (Section 7.4). We therefore describe no figure in this section as out of sample, as validated on test data, or as evidence of generalisation. The grids, the feasibility rule, the objective and the tie-break were fixed before the grids were run, at comparable density across the three shapes, with exact integer microseconds for arrivals, service and budget arithmetic. Supplementary Section S5 records what the selection chose, which results the released package reproduces and which remain archived development evidence, and the drop-one-cell sensitivity of the worst-cell rule. No sealed result is printed here.

8.3. Outcome Visibility in the Replayed Queue

Does the score a replayed job is ranked by use only results that would have been available in the replay? The original feature sweep releases an outcome at its source-platform done_j , whereas under policy P the replayed job completes at $a_j + W_P[j] + C_j$. A feature may therefore be causal in the recorded log and still use a result that is not available in the replay, and neither a future-record perturbation test nor a brute-force audit of the source-clock features resolves that. The exact protocol resolves it by construction. It freezes the fitted model, replays a policy, leaves out every same-copy result that the replay says was not complete when the job using it arrived, recomputes the affected histories and scores, and replays again, until no used result is unavailable. The set of left-out results only grows, so the procedure terminates, and every development cell ends with no used result unavailable. Two references sit beside it. The conservative score restricts user, exercise and pair

history to submission outcomes in the same class-term copy, admits a result only when $\text{done}_j + 3600 \leq a_i$, and refits its models under those restrictions, so it leaves out more than the replay requires. The static score uses code, course and clock information known at submission, without outcome history. Guard parameters remain those selected under the original score in every protocol. A lag audit of the conservative reference finds 0 violations in every conservative Guard and FCFS cell, and 336 job occurrences beyond the lag under unguarded conservative SPJF-E. Supplementary Section S3.7 gives the audit, the same-copy exposure decomposition, the aging comparator outside the budget family and the package predictor metrics with their splits.

At the busiest load, Guard(600) under the exact protocol gives p99 118.94 s and closes 0.672 [0.602, 0.743] of the reference gap, against 89.24 s and 0.801 [0.764, 0.842] under the original score and 174.71 s and 0.429 [0.357, 0.533] under the conservative score (Table 3). Unguarded SPJF-E closes 0.848 under the exact protocol and 0.670 under the conservative one. The static control closes 0.349 [0.281, 0.457]. The conservative figure can be taken apart: applying its three restrictions to the frozen original model gives 0.536, and refitting the models under them accounts for the rest of the fall to 0.429. Of the fall from 0.801, the restrictions account for 0.265 and the refit for 0.107. Leaving out, in addition, the results of other classes in the same term moves the exact figure to 0.655. These intervals condition on the constructed development overlays and do not cover semester selection, model refitting or deployment variation.

Table 3. Guard(600) under unchanged parameters and four information protocols. Exact is the headline: every score uses only results available in the replay. Original uses the source platform's own result clock and is an optimistic reference; conservative and static leave out more than the replay requires. Gap intervals are paired week-block intervals. Waits, excess and harm are in seconds.

Load	Score history	p99 _{dl}	Gap closed	Max exc.	Harm	Fired (%)
$\rho = 0.5, k = 7.8$						
	exact	15.25	0.724 [0.697, 0.747]	498.6	248.1	0.45
	original	14.97	0.740 [0.713, 0.760]	492.2	237.0	0.17
	conservative	16.68	0.640 [0.604, 0.671]	499.9	235.2	0.94
	static	17.95	0.560 [0.519, 0.594]	498.0	225.1	0.38
$\rho = 0.8, k = 5$						
	exact	51.90	0.785 [0.748, 0.807]	509.0	215.6	2.64
	original	47.47	0.837 [0.815, 0.851]	507.8	319.6	1.31
	conservative	64.77	0.634 [0.560, 0.688]	509.4	327.7	3.26
	static	70.18	0.571 [0.504, 0.638]	512.9	341.6	3.89
$\rho = 1, k = 4$						
	exact	118.94	0.672 [0.602, 0.743]	519.9	332.8	7.82
	original	89.24	0.801 [0.764, 0.842]	523.7	339.9	7.15
	conservative	174.71	0.429 [0.357, 0.533]	527.8	303.5	7.71
	static	193.11	0.349 [0.281, 0.457]	528.1	316.1	11.18

8.4. What to Rank By

Supplementary Table S16 gives the unguarded policies at the three loads. Ranking by expected cost is ahead of the log-scale score at the two higher loads and open at the lightest: the paired difference in gap closed is $-0.000 [-0.016, 0.013]$ at $\rho = 0.5$, $+0.012 [0.004, 0.019]$ at $\rho = 0.8$ and $+0.019 [0.011, 0.024]$ at $\rho = 1.0$. The control for refit noise, the same log-scale estimand refitted from scratch, moves the gap by $+0.001 [-0.002, 0.002]$, $-0.000 [-0.001, 0.001]$ and $-0.000 [-0.001, 0.000]$, so the advantage is larger than the noise of refitting and is not produced by it. That control and the sequence state come from the earlier development study and are the two rows of Supplementary Table S16 the package

does not produce. The sequence state of Section 4 does not translate into a consistent gain, and we do not claim it as an improvement.

Alternative ranking losses fitted on the same features and run through the same simulator are Supplementary Table S14: the fitted quantile scores and the raw-second squared-error model lose to the Tweedie fit at every load, and a two-class rule that only separates predicted-heavy from predicted-light jobs recovers little of the benefit, so most of it comes from the fine ordering inside the light class.

The two kinds of error do not cost the same. We find that the damage is concentrated in a few underestimates. Replacing the largest 1% of underestimates by the true cost, and leaving the rest of the score alone, closes 0.91 of the distance to the SJF reference in the deadline-window p99 at $\rho = 1.0$, on both overlays. Repairing the largest 1% of overestimates instead closes only 0.26 and 0.24. At 0.1% the two figures are 0.47 and 0.04 (Supplementary Section S3.16 gives the six waits behind them). Treating a heavy job as light and letting it run first is the expensive mistake, by about an order of magnitude. This identifies the fitted predictor: replacing a score by the true cost depends on its numerical scale and is not invariant to a monotone score transformation. It neither establishes a universal error asymmetry nor distinguishes the population targets of raw squared error and Tweedie loss.

8.5. What the Guarantee Costs and Gains

Table 4 reports the guarded policies at three promises and three loads, and Supplementary Figure S2 plots the gap closed against the promise. The cost of the promise depends on the promise and on the load. At $\rho = 1.0$, Guard(600) closes 0.801 of the FCFS-to-SJF gap against 0.916 for the unguarded policy. It therefore gives up 0.115 of the gap at a promise of ten job-limits, about an eighth of what the unguarded ordering gains. At $G = 1200$ s the cost is 0.087, and at $G = 300$ s, five job-limits, it is 0.438. The paired week-block intervals for these three differences are $[-0.142, -0.077]$, $[-0.098, -0.064]$ and $[-0.526, -0.314]$. None contains zero, so the promise has a resolved cost at every G we tested. All three use the original source-clock score, an optimistic reference rather than the headline of Table 3.

Table 4. The nine Guard(G) rows, at three promises and three loads, with the first-come first-served and SPJF-E figures of each block in its header. Guard(G) is the shape the selection rule picks over all three grids, and budgets scale as $k/4$. Supplementary Table S6 prints the same table in full, with Guard-fixed(G), the equal-promise Fixed(G) and the skip-count baseline at every promise, and carries the skip-count arithmetic. Excess and harm are the worst over the five overlays, the rest are means. All rows use original source-clock scores (optimistic reference).

Load	Policy	Promise	p99 _{dl}	Gap closed	Red. (%)	Max exc.	Harm	Fired (%)
$\rho = 0.5, k = 7.8$; FCFS p99 _{dl} = 27.64 s, SPJF-E 14.68 s with max excess 1,241.4 s								
	Guard(300)	300	15.58	0.705 [0.678, 0.724]	43.7	192.5	132.1	1.35
	Guard(600)	600	14.97	0.740 [0.713, 0.760]	45.8	492.2	237.0	0.17
	Guard(1200)	1200	14.84	0.746 [0.719, 0.768]	46.2	907.8	94.3	22.01
$\rho = 0.8, k = 5$; FCFS p99 _{dl} = 119.35 s, SPJF-E 43.33 s with max excess 3,044.5 s								
	Guard(300)	300	55.94	0.738 [0.697, 0.761]	53.0	209.7	114.1	11.57
	Guard(600)	600	47.47	0.837 [0.815, 0.851]	60.1	507.8	319.6	1.31
	Guard(1200)	1200	46.85	0.844 [0.823, 0.857]	60.6	1,104.9	117.6	9.93
$\rho = 1.0, k = 4$; FCFS p99 _{dl} = 273.50 s, SPJF-E 62.79 s with max excess 5,904.5 s								
	Guard(300)	300	163.40	0.478 [0.387, 0.602]	40.2	229.1	134.8	28.64
	Guard(600)	600	89.24	0.801 [0.764, 0.842]	67.3	523.7	339.9	7.15
	Guard(1200)	1200	82.67	0.830 [0.801, 0.857]	69.7	1,110.4	144.8	6.21

The validation harm constraint need not transfer to new cells. The original-score Guard(600) has evaluation harms of 319.6 and 339.9 s that exceed the 300 s validation limit

at the middle and busiest loads. This is failure of that empirical constraint to transfer, while every excess remains within the proved promise. Supplementary Figure S1 displays each selected shape and the full-cap comparator against harm, with the validation trade-offs shown separately, and Supplementary Section S3 makes the comparison with the aging baseline under the same conservative protocol.

A maximum waiting time keeps a promise of the same form (Proposition 1), so the budget counter has to earn its place against it. Table 5 runs Timeout($G - (3 - 2/k)L$) beside Guard(G) on the same five overlays and the same scores, so both make the promise G . At $\rho = 0.5$ they differ by at most 0.053 of the gap. Near saturation they part. At $\rho = 1.0$ and $G = 600$ s the clock closes 0.420 of the gap and the guard 0.801; at $G = 300$ s the clock closes 0.031, and its deadline-window p99 of 266.4 s is barely below the 273.5 s of FCFS. Both paired differences exclude zero. At $G = 1200$ s the two agree again, with a paired difference of +0.003 [-0.015, +0.031], and at the two lighter loads the clock is ahead by 0.006 and 0.013. The clock also harms more: its worst harm at $\rho = 1.0$ and $G = 600$ s is 456.7 s, against 339.9 s for the guard. What the counter buys is the tight promise near saturation, where the clock falls back to arrival order.

Table 5. Guard(G) against the maximum waiting time Timeout($G - (3 - 2/k)L$), which makes the same promise G (Proposition 1). Gap closed at the deadline-window p99, mean over the five overlays, with paired week-block intervals; the difference is Guard minus Timeout, resampled as a difference. Harm is the worst over the overlays, in seconds. Original source-clock scores, as in Table 4.

Load	G (s)	Guard(G)	Timeout	Difference	Harm, Guard / Timeout
0.5	300	0.705 [0.678, 0.724]	0.652 [0.619, 0.679]	+0.053 [+0.030, +0.077]	132.1 / 148.0
	600	0.740 [0.713, 0.760]	0.734 [0.707, 0.753]	+0.006 [+0.001, +0.012]	237.0 / 237.0
	1200	0.746 [0.719, 0.768]	0.752 [0.724, 0.774]	-0.006 [-0.009, -0.001]	94.3 / 237.0
0.8	300	0.738 [0.697, 0.761]	0.244 [0.017, 0.508]	+0.494 [+0.241, +0.691]	114.1 / 177.1
	600	0.837 [0.815, 0.851]	0.789 [0.754, 0.807]	+0.048 [+0.030, +0.080]	319.6 / 443.7
	1200	0.844 [0.823, 0.857]	0.857 [0.837, 0.869]	-0.013 [-0.017, -0.009]	117.6 / 469.2
1.0	300	0.478 [0.387, 0.602]	0.031 [0.012, 0.055]	+0.447 [+0.369, +0.550]	134.8 / 182.3
	600	0.801 [0.764, 0.842]	0.420 [0.131, 0.673]	+0.381 [+0.158, +0.650]	339.9 / 456.7
	1200	0.830 [0.801, 0.857]	0.827 [0.786, 0.865]	+0.003 [-0.015, +0.031]	144.8 / 708.8

Ordering by predicted cost inside fixed windows, as in the closest judging-queue scheduler [53], limits overtaking by position. We ran a stand-in for it on overlay 0: ranks are cut into consecutive blocks of W jobs, a job may start only when no earlier block has a waiting job, and the smallest predicted cost goes first inside the block. With blocks of up to 512 jobs it closes at most 0.417, 0.146 and 0.057 of the gap at the three loads, with worst excess 232.7, 359.6 and 444.5 s. Guard(300) closes 0.678, 0.716 and 0.486 on the same overlay, with worst excess 189.3, 199.7 and 219.6 s. A window bounds how far a job can be passed in positions, and near saturation a window large enough to help lets the long jobs wait longer than the guard does.

The jobs that pay are the heavy ones, and the worst wait among them is the last of the secondary metrics of Section 3. The unguarded ranking's worst excess of 5,904.5 s at the busiest load is the hundred job-limits of the introduction: 98.4 times the 60-second limit, for one job. The guard leaves the worst heavy job inside the promise at every load, within about 450 s of the FCFS figure, where the unguarded ranking multiplies it by between 2.8 and 4.1; Supplementary Section S3.16 gives the nine waits.

What the shape of the budget decides.

The promise is the cap's alone, so the three shapes are compared at a fixed promise, and the question is only how much of the base policy's benefit each keeps and who pays

for it. The shape matters where the promise is tight and the load is high, and hardly at all elsewhere. It decides which jobs are delayed more than it decides how much of the benefit survives. Supplementary Section S3 gives the paired differences at the three promises, the harm axis on which the queue-length shape comes within 0.009 of the joint winner's gap at under half its worst harm, and the equal-promise Fixed(G) comparator that the harm rule rejects at every promise while leading on the gap in eight of the nine cells.

Counting positions gains almost nothing on this load (Supplementary Figure S2, dotted). This measures how loose the count bound is here, not whether a count can give a bound at all. The conversion of Supplementary Section S4.4 does give one, and the count rule is run at the largest count that bound allows at each promise. What makes it loose is the cost distribution. Overtakers on this trace are overwhelmingly shorter than a second, while a count of κ has to be sized against the worst case κL . The count is therefore exhausted long before the work it stands for is, and the baseline forces most dispatches back to the queue head. It is charged at dispatch; Supplementary Section S3.16 gives the passes that allows and what charging at completion instead would change.

Corrupted scores separate what the guard does from what it does not do (Supplementary Table S8, Supplementary Figure S3). Under a reversed order, a random order, or a score that calls the truly longest 1% the shortest, the unguarded policy delays some job by more than 6,000 s beyond FCFS at full load, and Guard(600) holds every job inside its promise. The deadline-window p99 is not rescued: it stays at 546.5, 261.9 and 543.6 s against 273.50 s for FCFS, which is worse than FCFS for two of the three. The guarantee is per job and additive, so this is what it permits: a guarded run may have a worse quantile than FCFS, by at most G , and what it removes is the worst single job and not the shape of the tail.

8.6. The Bounds on the Traces

The promise of every bounded policy is asserted on every job of every run, inside the simulation rather than afterwards on summary statistics, so a violation would be a failure of the code and not of the theorem. That is 5,554,949,400 per-job checks over the 315 policy-by-cell runs reported here, 67,396,287 more in the single-server configuration and 5.7 billion in an independently written second simulator, with 0 violations anywhere. Propositions 1 and 2 were checked job by job after the runs on the same overlays, also with 0 violations, and Proposition 3 on constructed instances only (Supplementary Section S4.12). The ratio of the realised worst excess to the bound falls as k grows, from 0.9950 at $k = 1$ to 0.8850 at $k = 8$, while the finite-skip bound is attained exactly on small instances. The residual of Theorem 1 never exceeds 0.518 of its own ceiling $2(k - 1)L$, and Table 6 reports it on four cells. Supplementary Sections S3.13 and S3.16 give the identity measured job by job, the cross-check against the second simulator, the ratio at each k , and why a search that does not build the cascade of Theorem 2 stops where it does.

The absolute bound of Proposition 2 holds on every job of the 304 bounded runs as well, the first-come first-served and work-budget runs among them, with 0 violations. It comes close. At one server every first-come first-served wait equals its fluid backlog to the microsecond, as the proposition predicts, and the largest wait reaches 0.999 of the bound in its σ form, $(\sigma + B_{\max})/k + (2 - 1/k)L$. At the three loads the worst wait under a work budget reaches 0.886, 0.951 and 0.963 of its bound, and under first-come first-served 0.881, 0.945 and 0.965. An operator who quotes an absolute promise in advance needs a forecast of σ . The burst of the validation terms predicts that of the evaluation terms only to within a factor of 0.738 to 1.573, and it is too low in 10 of the 15 cells. A promise quoted from it is broken in 18 of 60 runs, by up to 349.8 s, so the absolute promise needs either a margin, here a factor of 1.36 on σ , or a shaper that holds the arrivals to the quoted bucket.

Table 6. The residual D_i of Theorem 1 on overlay 0, 17,634,760 jobs per cell. Ratio is $\max |D_i|$ over the bound $2(k-1)L$, which is $14L$, $8L$ and $6L$ at the three loads; R^2 and the worst error are for $(\text{In}_i - \text{Out}_i)/k$ against the realised excess over FCFS. The last row is the reversed score at the busiest load, against the Guard(600) row above it: the residual is a property of the schedule and barely moves with the policy. Supplementary Table S9 gives all nine cells.

Load	Policy	$\max D_i /L$	Ratio	$D_i = 0$	R^2	worst err. (s)
$\rho = 0.5, k = 8$	Guard(600)	6.788	0.485	95.6%	0.983	50.9
$\rho = 0.8, k = 5$	Guard(600)	3.922	0.490	87.2%	0.995	47.1
$\rho = 1.0, k = 4$	Guard(600)	3.082	0.514	81.1%	0.998	46.2
$\rho = 1.0, k = 4$	Reversed score	2.994	0.499	81.7%	1.000	44.9

8.7. A Single Server

At $k = 1$ the additive constant $(3 - 2/k)L$ is exactly L , the identity of Section 6 holds with no error term, and the separation between budget shapes is largest (Supplementary Table S10). Guard(600) and Fixed(600) close 0.854 and 0.868 of the gap, with overlapping intervals. They differ by a factor of 3.41 in harm, and the same comparison gives 3.01 at $G = 300$ s and 12.18 at $G = 1200$ s, so the looser the promise, the more the shape of the budget matters. The realised excess reaches 595.5 s against the promised 600 s, and under the worst corrupted score 597.0 s. At one server almost all of the promise is used.

The single-server configuration also shows the cost of choosing a shape on one set of cells: the parameters were selected on the 15 multiserver validation cells, and $k = 1$ is not among them, so the joint winner transfers at the two tighter promises and not at $G = 1200$ s (Supplementary Section S3.14). We report the number the pre-stated rule produces and do not re-tune it on this cell; Section 9 states what it means.

8.8. Other Platforms

Table 7 places the main trace beside the other scheduling workloads under one format, including the supplementary LPC-EGEE [83] counterfactual. The first three columns describe the workload, the next two what ranking by prediction does to it without a guard, and the last two the gap the guard closes and the excess it caps at a promise of five job-limits. Excesses are expressed in units of the per-job limit L , the unit the guarantee is written in.

Two empirical screens say where reordering and the uniform certificate may be useful. The first compares the FCFS tail scale with the certificate's additive term $(3 - 2/k)L$. Four of the original six workloads have smaller FCFS p99 values. That means the certificate is too coarse at that scale, not that finer promises are impossible for a restricted base policy or workload. CodeBench at $4.56L$ and the serverless trace at $4.22L$ clear it. The compute farm is the extreme, with an FCFS p99 of 18.7 h against a bound $L = 493,857$ s, or 5.7 days, and guards that never fire. The second screen is cost concentration, an opportunity for reordering rather than a necessary condition. In the CI pools the longest 1% carry only 3.4% and 5.2% of work, and true-size ordering makes p99 worse on three of the four reported windows. Where both screens are cleared, in the original-score CodeBench and serverless comparisons, the guard closes 0.48 to 0.71 of the reference gap and limits observed excess to 2.5 to $3.8L$ at a promise of $5L$. Clearing either screen does not guarantee a percentile improvement: the capacity experiment below contains counterexamples.

Which screen a trace clears can depend on how its pool is drawn. Read as one pool per walltime class rather than one per site, LPC-EGEE's 900-second class has an FCFS p99 of $13.76L$. It is the one reading of that log on which the work-budget guard fires, spending 0.993 of its allowance against 0.046 pooled. Cost concentration inside it falls to 0.213 from

Table 7. Cross-domain development comparisons. Tail share is the fraction of executed work carried by the longest 1% of jobs; AUROC is for the heavy class from arrival-time information; excesses are in units of the per-job limit L of that trace. “Cond. 1” is the first applicability condition of this section, that the first-come first-served p99 wait exceed the floor $(3 - 2/k)L$, which is close to $3L$: four of the original six workloads fail it, and on those four the uniform certificate is coarser than the wait itself. This is a usefulness screen for that certificate, not an impossibility claim for other policies. The CodeBench row uses original source-clock scores (optimistic reference).

Trace and load	Tail share	AUROC	FCFS p99/ L	Cond. 1?	Gap SPJF-E	Unguarded max exc.	Guard at $G \approx 5L$ gap closed / max exc.
CodeBench, busy-hour $\rho \approx 1.0$	0.375	0.936	4.56	yes	0.916	98.4 L	0.478 / 3.82 L
ACcoding, drawn arrivals, $\rho = 0.8$	0.122	0.927	0.021	no	0.980	20 L	0.974 / 2.21 L ^a
Serverless, busy-hour $\rho = 1.0$	0.443	0.985	4.22	yes	0.858	10.07 L	0.710 / 2.52 L
CI pool A ^c , $k_{\text{eff}} = 156$ of 173	0.034	0.651	1.78	no	n/a ^b	11.53 L	0.735 ^b / 2.64 L
CI pool B ^c , $k_{\text{eff}} = 96$ of 105	0.052	0.997	0.373	no	0.958	0.34 L	0.958 / 0.34 L
Compute farm, window $\rho = 0.95$	0.372	0.890	0.136	no	−4.111	0.85 L	−4.346 / 0.83 L
LPC-EGEE, chronological test ^d	0.351	0.892	0.042	no	0.193	0.162 L	0.193 / 0.162 L
LPC-EGEE test class, per-class pool ^e	0.213	0.746	13.76	yes	0.679	86.07 L	0.569 / 3.48 L

^a ACcoding arrival instants are drawn, not recorded, so every scheduling figure in this row is a statement about the ordering mechanism and not about waiting on that platform; its budgets were set at fixed multiples of L instead of from a promise, and the entry shown is the largest one tested, $B = 10L$. Its tail share is the simulated one, with the assumed fixed term and $L = 30.5$ s; on measured cost alone, at $L = 30.0$ s, it is 0.275 (Supplementary Table S3), the figure the introduction quotes. ^b On CI pool A, SJF is itself worse than FCFS at the p99, so the p99 gap has no usable denominator; the guard entry is the mean-wait gap. ^{c,d,e} What the fitted continuous-integration capacities, the LPC-EGEE chronological test and the imposed test-class pool do and do not support is stated in Supplementary Section S6.7.

the 0.407 of the pooled trace. The row’s footnote and Supplementary Section S6.6 say why that is not evidence of deployment benefit.

8.9. Capacity and Physical Implementation

Benefit changes sign with capacity. Across every integer $k = 1, \dots, 20$ on the five development overlays, Guard(600) makes deadline p99 worse by 493.79 s at $k = 1$, and positive simultaneous support for its improvement occurs only at $k = 4$ –12. The selected load points are therefore not evidence of positive benefit at every capacity. The physical timed-work experiment executes 2,151 jobs on two workers in a selected empty-start release window: all 6,453 observed-prefix decisions pass the rule audit, but the guard’s per-job difference from its ideal measured-input replay reaches 1,354.10 s. Its physical p99 is 1,783.65 s against 1,763.30 s for separately timed FCFS, so this window does not establish a guarded p99 gain. Supplementary Sections S6.1 and S6.2 give the sweep protocol, the selection rule and the positive execution checks beside these negative results.

A second physical run puts the guard on the two-worker back end of a course platform, whose jobs are calls to a language model that write a study plan or a learner report, and whose server stops a job at an enforced limit of 170 s or 25 s by class. Twenty scripted accounts submitted 100 jobs per cell in a deadline-shaped burst, each waiting for its result before submitting again, under a promise of 400 s. Unguarded predicted-order dispatch left 7 jobs beyond the promise, the worst by 4.67 times G . The four guarded cells, 400 jobs in all, left none, with a worst excess of 0.514 G , and the package simulator, fed the measured arrivals and service times, reproduced every measured wait within 0.17 s in the five cells it can simulate; it has no reservation-and-refund wrapper for the sixth. Because the accounts waited for results, the arrival sequence differed from cell to cell, so each cell is compared only with its own replayed first-come first-served run. One hundred jobs per cell give no p99, and no human learner was in the loop. Supplementary Section S6.9 gives the six cells, the enforced-limit check, the dispatch overhead and what the run does not support.

8.10. Sensitivities

Five trace-construction variants of an earlier source-clock pipeline are a sensitivity to that pipeline and not to the selected Guard(G); Supplementary Table S15 and Supplementary Section S3.17 give them, and Supplementary Section S5 gives the configuration and the release boundary. A sixth variant changes the arrival process instead of the trace construction. The logs carry a recorded think time, the offset between one result and that student's next submission. Releasing each submission after its predecessor's result plus that offset shortens the deadline-window p99 by a factor of 1.3 to 3.8, and the mean wait by 1.5 to 3.9. The baseline gains most, because its own longer waits delay its own users more. The absolute gap therefore narrows while the fraction of it a policy closes does not: Guard(600) closes 0.825, 0.809 and 0.810 of it, against 0.734, 0.827 and 0.800 at the recorded instants. The cost of the promise falls with the queue, from 41.6% above unguarded SPJF-E to 3.7%, and the guarantee holds on all 17,634,760 jobs of each run against first-come first-served on that run's own realised arrivals. These runs read the original-clock scores of the main development cells, so they are a sensitivity to the arrival process and not a repeat of the exact-protocol headline. Supplementary Section S6.5 gives the table, the bootstrap and the limits, the first being that the recorded think times were produced on a platform where no student waited.

Two further limits of these runs — what the intervals do and do not cover, and the prediction cost the runs do not charge — are stated in Section 9.

9. Limitations and Threats to Validity

Three things here could be wrong in ways the experiments would not show. The queue we replay never existed. The scores we rank by were fitted with knowledge of later events that we only partly remove. And the guarantee is measured in L . We take them in turn, and the smaller threats after them.

The shared pool is our design.

CodeBench creates a separate container for each student [80], so on the real platform these logs do not identify a shared judging queue [16]. We use them as a demand trace, with reconstructed arrivals and recorded execution durations. The server pool, the queue and the scheduler are ours, and nothing here is a statement about waiting on the real platform. What supports the design is that a fixed pool of judge hosts on one queue is a standard deployment [1,2], and that pooling replaces 1,636 concurrent containers with 14 servers on the same demand [84]. Section 7 and Supplementary Section S2.3 give that evidence. They also say why the container ratio belongs to the overlay and to no deployment, and why it is not a cost saving, since the container also serves the student's editing session. The physical timed-work run does not validate transport of submitted-program costs either: its rule-conformance checks sit beside a worse guarded p99 and with large per-job deviations from the ideal simulator (Supplementary Section S6.2).

Open-loop replay.

Arrivals come from the log at their recorded instants and do not react to our scheduler. A scheduler is therefore neither rewarded with the work it would attract nor punished by the users it would drive away [85,86]. One component of that feedback is in the logs, which identify each submission's author and its position in that student's sequence. The offset between one result and the next submission is a recorded think time: it is non-negative for 98.38% of consecutive pairs, its median is 123.5 s, and its Spearman correlation with the previous job's cost is 0.015. We replayed the traces a second time with each submission released after its predecessor's result plus that offset. The deadline-window

p99 and the mean wait both shorten, and the benefit reverses in none of the nine development cells: on their original-clock scores Guard(600) closes 0.734, 0.827 and 0.800 of the FCFS-to-SJF deadline-window gap at the three loads under the recorded instants, and 0.825, 0.809 and 0.810 under gating (Supplementary Section S6.5). That replay does not restore abandonment or content feedback, and the recorded think times come from a platform on which no student ever waited. So the direction of what is left is still not identified. The results are conditional on fixed offered work rather than lower bounds on closed-loop benefit, and the exact feedback counterexample in Supplementary Section S6.3 shows a scheduling advantage disappearing in one construction and reversing in another. That section also states Feitelson's objection in his own words and the productivity distinction Shmueli and Feitelson draw. The width coupling that distorts parallel-job replays in a pool of another size does not arise here, because a job occupies one server whatever k is. One choice is deliberate. Trace-driven studies often raise the load by compressing inter-arrival times [87], which shrinks the daily load cycle along with the gaps [85]. We raise it instead by overlaying whole class-semesters aligned on weekday and clock time.

The guarantee is measured in L .

Every additive constant in Section 6 is a multiple of the per-job time limit, apart from the report delay δ and the arrival burst σ . The additive constant $(3 - 2/k)L$ is an exact supremum, not an artefact of the proof. It grows with k , from L at a single server towards $3L$. A promise that is to hold around every base policy cannot be less than L unless the wrapper gives up reordering altogether. The reason is simple. Whenever the budget is positive for the waiting jobs at the first dispatch epoch, there are a base policy and an input on which some job's excess is at least L . The only budget rules of this family that escape are the ones that make the wrapper first-come first-served itself (Proposition 4). So no mechanism of this family that reorders at all can promise less than $\Omega(L)$. The obvious escape from that floor is kill-and-restart tiering, and it fails. It is TAGS [88] with the dedicated hosts removed, and removing them is what breaks it. The kill becomes terminal, so the excess it leaves is wasted work rather than overtaking, and no budget removes it. The light-job constant does not improve either, because a second-tier piece is still a non-preemptive block of length L . Supplementary Section S7.5 gives both arguments with the constant we measure. When the service enforces a per-job cap that is known at arrival, charging the cap at dispatch and refunding it at completion lowers the constant by exactly L (Section 6); it does not remove the floor.

Two applicability conditions, each with traces that fail it.

The first condition asks whether the uniform certificate is informative at the waits that matter, and the second is an empirical opportunity screen rather than a theorem assumption. Four of the original six workloads fail the first, among them the Intel Netbatch compute farm [18,19], whose service cap of 5.7 days is the largest, and the two Firefox CI pools [20,21] fail the second. Section 8.8 gives both conditions in numbers, trace by trace, and Supplementary Section S6.8 says what failing either does and does not establish.

The guarantee is per job, not per quantile.

On a fixed set of jobs, the pointwise bound implies $Q_\alpha(W_P) \leq Q_\alpha(W_{\text{FCFS}}) + G$ by monotonicity of quantiles. It does not imply an improvement in that percentile. A guarded run can have a worse deadline-window p99 than the baseline its guarantee is stated against, by as much as the promise itself. Supplementary Table S8 shows it doing so under two of the three corrupted scores at the busiest load, while the worst single job stays inside the promise.

Development data, and what is still sealed.

Every CodeBench number in this paper is development data, and none of it is out of sample. We explored the guard family and an early parameter grid on the same overlay the tables report, and only then searched the three pre-stated grids on validation overlays that exclude the evaluation semester. The selection is therefore validation-only inside a family that is not. The rule is a worst-cell rule and is sensitive to which validation cells it sees: at the middle promise, dropping the one cell that decides the choice moves it to a different point of the same grid. The five overlays are week-shifts of the same class-terms rather than five independent tests. For both reasons we do not describe any of these parameters as validated. The guarantee does not depend on the parameters being well chosen; only the benefit does. The sealed semesters will be opened once, after the method, the parameters and the primary metric have been frozen, and their numbers reported separately. That evaluation is not completed here. The second judging platform's sealed id block and the corroborating trace's sealed year stay shut, and the frozen pipeline covers the main trace alone (Section 7.6).

The budget shape is selected on one set of cells, and does not always transfer.

The promise holds for any budget under the cap, so the shape decides only how much of the base policy's benefit survives, and we select it on 15 multiserver validation cells that do not include $k = 1$. Supplementary Section S3.14 gives the single-server configuration where that choice does not transfer, what the pre-stated rule reports there, and why an operator with a much smaller pool should re-run the selection instead of taking our shape.

What the recorded costs mean.

The cost we replay is the platform's recorded execution time, a wall-clock duration for one evaluation that already contains contention we then re-impose inside the pool. Supplementary Section S2.1 gives how much of it is contention, and why the observed 60-second regime is not documentation of an enforced timeout. It also records that the released ACcoding tables [17] carry no submission time, although the platform recorded one, so its arrival process is synthetic, and Supplementary Section S2.3 reports which quantile of the Firefox CI refit does not survive out of sample.

History availability and the recorded-queue counterexample.

Original-clock scores may use outcomes that are still pending in the replay. The exact protocol certifies that its final replay uses none, cell by cell, and it is the headline for that reason. It does not certify an online fixed point, in which a deployed predictor is driven by completion events as they happen, and it does not certify a deployment in which copied users share state across copies. The conservative reference is not a clean control for availability: it also reduces history and refits models, and about a quarter of its fall from the original figure comes from the refit (Section 8.3).

LPC-EGEE [83] provides both recorded waits and concentrated costs, but its fitted first-come first-served approximation underestimates the chronological-test mean waits, and the one reading on which a guard fires is a per-class pool we impose ourselves. We therefore treat its policy replays as conditional counterfactuals rather than validated estimates of deployment benefit, and Supplementary Sections S6.4 and S6.6 give both readings with the six objections the per-class one does not answer.

Not studied.

Preemption is outside the model, so nothing here applies to a system that can checkpoint or suspend. The wrapper applies unchanged to a learned or reinforcement-learned

base policy, since its bound assumes nothing about the base, but we did not test one. Capacity is fixed within each replay. The capacity sweep is a sensitivity to that choice rather than an estimate of the original platform's capacity, and it is not a capacity-planning study. The reported runs compare policies at a common set of enqueue instants and do not charge the cost of computing features and running the predictor. Section 4 measures that cost and charges it in a sensitivity, where it moves the closed gap by less than 0.001 at every load. What is missing is therefore not the latency but the rest of an end-to-end system. The local timed-work service was executed, but there was no prospective production deployment and no measurement of user feedback. ALPS [59], a prototype serverless scheduler of the kind this wrapper is meant for, has no proved per-job bound.

10. Conclusions

An operator can now answer the student whose submission sat behind the looping one. The queue is reordered by predicted cost, and no submission waits more than G seconds longer than it would have in arrival order, whatever the predictor does. We find that work, not position, is the unit the excess is written in. A job's delay relative to first-come first-served equals its net overtaken work divided by the number of servers, up to a constant multiple of the time limit, and a budget on that quantity is sufficient for a per-job guarantee and, up to an additive $O(kL)$, necessary. A maximum waiting time makes a promise of the same form, and at the busiest load and tight promises it keeps much less of the benefit, because it charges a job for the backlog ahead of it as well as for the jobs that pass it. Counting overtakes is not necessary, and it gives a guarantee only at the worst-case charge of the time limit per pass. On the main trace that charge is 150 times the mean job cost. With scores that use only results available in the replay, Guard(600) closes 0.672 of the reference gap at the busiest load, against 0.848 for unguarded ordering. These are development results, conditional on the constructed demand and on the information protocol. Four of the original six workloads have tail waits too short relative to L for the uniform certificate to be useful. Concentration of job costs is an empirical opportunity for reordering, and we do not claim it is necessary.

We have not run the guard in production. The timed-work service reproduces the dispatch rule but not the ideal simulator's waits, and the sealed terms are still sealed. The graph model's edges add a small rank-correlation gain over all test rows, 0.006, and a larger one, 0.028, on tasks with fewer than five prior results, 1,092 of the 40,844 test rows here. Two questions are left open. The identity is stated in the dispatch-sequence convention, where the constant is exactly determined in both directions. Under the executed-work convention the upward constant is unchanged, but we know only that the downward one lies in $[(k-1)L, 2(k-1)L]$; closing that gap would make the two conventions interchangeable. And Algorithm 1 is not claimed to be the least restrictive rule carrying its guarantee: whether some wrapper overrides the base policy strictly less often and still promises G is a question we have not settled in either direction.

Supplementary Materials: The following supporting information can be downloaded at <https://www.mdpi.com/article/10.3390/math1010000/s1>: Table S1: Predictors on the four non-judging traces, by feature group; Table S2: Predictors on the two judging traces under one visibility protocol and one split; Table S3: Share of executed work carried by the longest jobs, with the provenance of the analysis bound on every trace; Table S4: Experimental configuration and the selection protocol for the guard parameters; Table S5: The three selected budget points and the nine per-shape grid winners; Table S6: The guard at three promises and the comparators, in full; Table S7: The age-relative and queue-length winners at three promises and three loads; Table S8: Deliberately corrupted scores, unguarded and under the guard; Table S9: The residual of Theorem 1 on all nine cells; Table S10: The single-server configuration; Table S11: Additional policy definition for linear aging;

Table S12: Conservative-score comparison with the selected aging baseline; Table S13: Package development AUROC with user-block intervals; Table S14: Ranking scores fitted on one feature set and run through one simulator; Table S15: Trace-construction sensitivities; Table S16: Unguarded policies, five overlays; Table S17: Candidates and feasible points after promise assignment and the harm filter; Table S18: Capacity sweep with archived scores; Table S19: Physical timed-work runs; Table S20: Open-loop against think-time-preserving replay; Table S21: LPC-EGEE by walltime class, both partitions pooled; Table S22: Replay of the 900-second test class at its own limit; Table S23: Six cells on the platform back end; Table S24: Scope of the nearest per-job guarantees and wrapper precedent; Figure S1: Gap-harm trade-offs for every selected budget shape and Fixed(G); Figure S2: What each budget shape keeps of the FCFS-to-SJF gap as the promise varies; Figure S3: Original-score reference and its corrupted-score controls. The nine supplementary sections that carry them are Section S1, cost prediction, with the visibility audit in full, the release delay as a sensitivity, the predictor families and the measured cost of predicting; Section S2, traces, with the field audit of the main judging trace, the checks on the continuous-integration refit and what supports the shared-pool design; Section S3, experiments; Section S4, the deferred proofs, including Lemma 1, Theorem 1 with the two directions of its extremes, the consistency proposition with the two counterexamples to its converse, the multiplicative bound for the age-relative shape, the count-of-overtakes remark, the bound for a maximum waiting time, the absolute bound from the arrival log, the bound when completion reports arrive late, and the relaxations of the assumptions behind Section 6; Section S5, the reproducibility specification, with exact grids, splits, information clocks and table provenance; Section S6, the capacity sweep, the physical timed-work experiment, the feedback counterexamples, the closed-loop replay, the LPC-EGEE counterfactual and the footnotes to Table 7; Section S7, additional theory, with reservation-and-refund charging under enforced run-time caps; Section S8, the extremal constructions, together with the tightness of the guard bound, achievability and the maximal permission rule at one server; and Section S9, the extended related work behind Section 2.

Author Contributions: Conceptualization, [A.B.] and [C.D.]; methodology, [A.B.]; software, [A.B.]; validation, [A.B.] and [C.D.]; formal analysis, [A.B.]; investigation, [A.B.]; resources, [C.D.]; data curation, [A.B.]; writing—original draft preparation, [A.B.]; writing—review and editing, [A.B.] and [C.D.]; visualization, [A.B.]; supervision, [C.D.]; project administration, [C.D.]; funding acquisition, [C.D.]. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by [funding body to be completed by the authors], grant number [grant number to be completed by the authors].

Institutional Review Board Statement: Not applicable. This study used previously published, pseudonymised logs and involved no human participants and no animals.

Informed Consent Statement: Not applicable.

Data Availability Statement: The public source datasets used here are available at the locations cited, and we redistribute none of them. The CodeBench judging logs are published by the Universidade Federal do Amazonas [16]; the ACcoding dataset is published with its data descriptor [17]; the Open University Learning Analytics dataset is published with its own [79]; the Azure Functions 2021 invocation trace is published in the Azure Public Dataset repository [7], which asks that [8] be cited; and the Intel Netbatch logs are published in the Parallel Workloads Archive [18], which asks that the logs be acknowledged to their depositors and suggests citing [19]. The LPC-EGEE log is also published in the Parallel Workloads Archive [83], with acknowledgment requested for its data provider. The continuous-integration trace of the two Firefox hardware worker pools, was collected by us from Mozilla's public, unauthenticated Taskcluster and Treeherder APIs [20,21]; we do not redistribute it, and the collection scripts that rebuild it from those APIs are part of the released code. The Azure invocation trace is distributed under the publisher's Creative Commons Attribution licence [7]; public availability does not imply an absence of licence conditions. No restricted access agreement was used, and no personal data were processed beyond the pseudonymous user identifiers the published judging datasets already carry. The simulator, the predictors, the guard implementation, configurations and archived development analyses are to be released with the pa-

per. Supplementary Section S5 distinguishes the tables reproduced by the package from the earlier development evidence. The release will be available at [repository URL to be completed by the authors].

Acknowledgments: [to be completed by the authors].

Conflicts of Interest: The authors declare no conflicts of interest. [To be confirmed by the authors: if there is a funder, state here what role it had in the design of the study, in the collection, analysis or interpretation of data, in the writing of the manuscript, or in the decision to publish the results.]

Abbreviations

The following abbreviations are used in this manuscript:

AUPRC	Area under the precision–recall curve
AUROC	Area under the receiver operating characteristic curve
CI	Continuous integration
FCFS	First-come first-served
GRU	Gated recurrent unit
MLP	Multilayer perceptron
OULAD	Open University Learning Analytics Dataset
RMSE	Root mean squared error
SJF	Shortest job first
SPJF	Shortest predicted job first
SRPT	Shortest remaining processing time

References

- Peveler, M.; Maicus, E.; Cutler, B. Comparing Jailed Sandboxes vs Containers Within an Autograding System. In Proceedings of the 50th ACM Technical Symposium on Computer Science Education (SIGCSE '19), 2019, pp. 139–145. <https://doi.org/10.1145/3287324.3287507>.
- DOMjudge team. DOMjudge Administrator's Manual: Overview and Judging. <https://www.domjudge.org/docs/manual/main/overview.html>. Accessed 2026-09-19.
- Wasik, S.; Antczak, M.; Badura, J.; Laskowski, A.; Sternal, T. A Survey on Online Judge Systems and Their Applications. *ACM Computing Surveys* **2018**, *51*, 1–34. <https://doi.org/10.1145/3143560>.
- König, C.J.; Kleinmann, M. Deadline Rush: A Time Management Phenomenon and Its Mathematical Description. *The Journal of Psychology* **2005**, *139*, 33–45. <https://doi.org/10.3200/JRPL.139.1.33-45>.
- Alfi, V.; Parisi, G.; Pietronero, L. Conference registration: how people react to a deadline. *Nature Physics* **2007**, *3*, 746. <https://doi.org/10.1038/nphys761>.
- Alfi, V.; Gabrielli, A.; Pietronero, L. How people react to a deadline: time distribution of conference registrations and fee payments. *Open Physics* **2009**, *7*, 483–489. <https://doi.org/10.2478/s11534-009-0059-z>.
- Microsoft Azure. Azure Functions Invocation Trace 2021. <https://github.com/Azure/AzurePublicDataset/blob/master/AzureFunctionsInvocationTrace2021.md>, 2021.
- Zhang, Y.; Goiri, Í.; Chaudhry, G.I.; Fonseca, R.; Elnikety, S.; Delimitrou, C.; Bianchini, R. Faster and Cheaper Serverless Computing on Harvested Resources. In Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP '21), 2021, pp. 724–739. <https://doi.org/10.1145/3477132.3483580>.
- Scully, Z.; Grosz, I.; Mitzenmacher, M. Uniform Bounds for Scheduling with Job Size Estimates. In Proceedings of the 13th Innovations in Theoretical Computer Science Conference (ITCS 2022). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2022, Vol. 215, *LIPICs*, pp. 114:1–114:30. Preprint arXiv:2110.00633, <https://doi.org/10.4230/LIPICs.ITCS.2022.114>.
- Mitzenmacher, M.; Shahout, R. Queueing, Predictions, and Large Language Models: Challenges and Open Problems. *Stochastic Systems* **2025**, *15*, 195–219. Preprint arXiv:2503.07545, <https://doi.org/10.1287/stsy.2025.0106>.
- Zhao, T.; Xia, C.; Chang, X.; Li, C.; Li, W.; Zomaya, A. Competitive Fair Scheduling with Predictions. In Proceedings of the The Thirteenth International Conference on Learning Representations (ICLR 2025), 2025.
- Yu, G.; Scully, Z. Strongly Tail-Optimal Scheduling in the Light-Tailed M/G/1. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* **2024**, *8*, 1–33. Preprint arXiv:2404.08826, <https://doi.org/10.1145/3656011>.
- Yu, G.; Harlev, A.; Adakroy, R.; Scully, Z. A Tale of Two Traffics: Optimizing Tail Latency in the Light-Tailed M/G/k. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* **2025**, *9*, 1–40. <https://doi.org/10.1145/3771561>.

14. Ke, G.; Meng, Q.; Finley, T.; Wang, T.; Chen, W.; Ma, W.; Ye, Q.; Liu, T.Y. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In Proceedings of the Advances in Neural Information Processing Systems 30 (NeurIPS 2017), 2017, pp. 3146–3154.
15. Schlichtkrull, M.; Kipf, T.N.; Bloem, P.; van den Berg, R.; Titov, I.; Welling, M. Modeling Relational Data with Graph Convolutional Networks. In Proceedings of the The Semantic Web (ESWC 2018), 2018, Vol. 10843, *Lecture Notes in Computer Science*, pp. 593–607. Preprint arXiv:1703.06103, https://doi.org/10.1007/978-3-319-93417-4_38.
16. Universidade Federal do Amazonas, Instituto de Computação. CodeBench Dataset, version 1.81. <https://codebench.icomp.ufam.edu.br/dataset/>. Accessed 2026-09-19.
17. Chen, K.; Huang, F.; Liu, Z.; Yu, H.; Meng, L.; Mo, S.; Zhang, L.; Song, Y. ACcoding: A graph-based dataset for online judge programming. *Scientific Data* **2024**, *11*. See also Author Correction, doi:10.1038/s41597-024-03693-3, <https://doi.org/10.1038/s41597-024-03392-z>.
18. Parallel Workloads Archive. The Intel Netbatch Logs. https://www.cs.huji.ac.il/labs/parallel/workload/I_intel_netbatch/, 2012.
19. Shai, O.; Shmueli, E.; Feitelson, D.G. Heuristics for Resource Matching in Intel's Compute Farm. In Proceedings of the Job Scheduling Strategies for Parallel Processing (JSSPP 2013). Springer, 2014, Vol. 8429, *Lecture Notes in Computer Science*, pp. 116–135. https://doi.org/10.1007/978-3-662-43779-7_7.
20. Mozilla. Taskcluster Documentation: Queue Service Reference. <https://docs.taskcluster.net/docs/reference/platform/queue>. Accessed 2026-09-19.
21. Mozilla. Treeherder REST API Documentation. <https://treeherder.mozilla.org/docs/>. Accessed 2026-09-19.
22. Purohit, M.; Svitkina, Z.; Kumar, R. Improving Online Algorithms via ML Predictions. In Proceedings of the Advances in Neural Information Processing Systems 31 (NeurIPS 2018), 2018.
23. Mitzenmacher, M. Scheduling with Predictions and the Price of Misprediction. In Proceedings of the 11th Innovations in Theoretical Computer Science Conference (ITCS 2020). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2020, Vol. 151, *LIPICs*, pp. 14:1–14:18. Preprint arXiv:1902.00732, <https://doi.org/10.4230/LIPICs.ITCS.2020.14>.
24. Azar, Y.; Leonardi, S.; Tseitou, N. Flow Time Scheduling with Uncertain Processing Time. In Proceedings of the 53rd Annual ACM SIGACT Symposium on Theory of Computing (STOC), 2021, pp. 1070–1080. <https://doi.org/10.1145/3406325.3451023>.
25. Azar, Y.; Leonardi, S.; Tseitou, N. Distortion-Oblivious Algorithms for Minimizing Flow Time. In Proceedings of the 2022 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA), 2022, pp. 252–274. Preprint arXiv:2109.08424, <https://doi.org/10.1137/1.9781611977073.13>.
26. Dong, J.; Ibrahim, R. Shortest-Job-First Scheduling in Many-Server Queues with Impatient Customers and Noisy Service-Time Estimates. *Operations Research* **2025**, *73*, 3139–3155. <https://doi.org/10.1287/opre.2022.310>.
27. Shahout, R.; Mitzenmacher, M. SkipPredict: When to Invest in Predictions for Scheduling. In Proceedings of the Advances in Neural Information Processing Systems 37 (NeurIPS 2024), 2024, pp. 105663–105701. Preprint arXiv:2402.03564, <https://doi.org/10.52202/079017-3352>.
28. Schwiegelshohn, U.; Yahyapour, R. Fairness in Parallel Job Scheduling. *Journal of Scheduling* **2000**, *3*, 297–320. [https://doi.org/10.1002/1099-1425\(200009/10\)3:5<297::AID-JOS50>3.0.CO;2-D](https://doi.org/10.1002/1099-1425(200009/10)3:5<297::AID-JOS50>3.0.CO;2-D).
29. Sabin, G.; Kochhar, G.; Sadayappan, P. Job Fairness in Non-Preemptive Job Scheduling. In Proceedings of the 2004 International Conference on Parallel Processing (ICPP 2004). IEEE Computer Society, 2004, pp. 186–194. <https://doi.org/10.1109/ICPP.2004.1327920>.
30. Sabin, G.; Sadayappan, P. Unfairness Metrics for Space-Sharing Parallel Job Schedulers. In Proceedings of the Job Scheduling Strategies for Parallel Processing (JSSPP 2005), 2005, Vol. 3834, *Lecture Notes in Computer Science*, pp. 238–256. https://doi.org/10.1007/11605300_12.
31. Rajbhandary, A.; Bunde, D.P.; Leung, V.J. Variations of Conservative Backfilling to Improve Fairness. In Proceedings of the Job Scheduling Strategies for Parallel Processing (JSSPP 2013). Springer, 2014, Vol. 8429, *Lecture Notes in Computer Science*, pp. 177–191. https://doi.org/10.1007/978-3-662-43779-7_10.
32. Mu'alem, A.W.; Feitelson, D.G. Utilization, predictability, workloads, and user runtime estimates in scheduling the IBM SP2 with backfilling. *IEEE Transactions on Parallel and Distributed Systems* **2001**, *12*, 529–543. <https://doi.org/10.1109/71.932708>.
33. Lindsay, A.M.; Galloway-Carson, M.; Johnson, C.R.; Bunde, D.P.; Leung, V.J. Backfilling with guarantees made as jobs arrive. *Concurrency and Computation: Practice and Experience* **2013**, *25*, 513–523. <https://doi.org/10.1002/cpe.2860>.
34. Talby, D.; Feitelson, D.G. Supporting Priorities and Improving Utilization of the IBM SP Scheduler Using Slack-Based Backfilling. In Proceedings of the 13th International Parallel Processing Symposium and 10th Symposium on Parallel and Distributed Processing (IPPS/SPDP 1999), 1999, pp. 513–517. <https://doi.org/10.1109/IPPS.1999.760525>.
35. Yuan, Y.; Yang, G.; Wu, Y.; Zheng, W. PV-EASY: A Strict Fairness Guaranteed and Prediction Enabled Scheduler in Parallel Job Scheduling. In Proceedings of the 19th ACM International Symposium on High Performance Distributed Computing (HPDC '10), 2010, pp. 240–251. <https://doi.org/10.1145/1851476.1851505>.

36. Yuan, Y.; Wu, Y.; Zheng, W.; Li, K. Guarantee Strict Fairness and Utilize Prediction Better in Parallel Job Scheduling. *IEEE Transactions on Parallel and Distributed Systems* **2014**, *25*, 971–981. <https://doi.org/10.1109/TPDS.2013.88>.
37. Grosz, I.; Scully, Z.; Harchol-Balter, M. Load Balancing Guardrails: Keeping Your Heavy Traffic on the Road to Low Response Times. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* **2019**, *3*, 1–31. Preprint arXiv:1905.03439, <https://doi.org/10.1145/3341617.3326157>.
38. Ramadge, P.J.; Wonham, W.M. Supervisory Control of a Class of Discrete Event Processes. *SIAM Journal on Control and Optimization* **1987**, *25*, 206–230. <https://doi.org/10.1137/0325013>.
39. Grosz, I.; Yang, K.; Scully, Z.; Harchol-Balter, M. Nudge: Stochastically Improving upon FCFS. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* **2021**, *5*, 1–29. Preprint arXiv:2106.01492, <https://doi.org/10.1145/3460088>.
40. Van Houdt, B. On the Stochastic and Asymptotic Improvement of First-Come First-Served and Nudge Scheduling. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* **2022**, *6*, 1–22. Preprint arXiv:2206.10428, <https://doi.org/10.1145/3570610>.
41. Charlet, N.; Van Houdt, B. Tail Optimality and Performance Analysis of the Nudge*(M) Scheduling Algorithm. *Operations Research* **2026**, *74*, 2161–2178. <https://doi.org/10.1287/opre.2024.1061>.
42. Grosz, I.; Harchol-Balter, M.; Scheller-Wolf, A. WCFS: a new framework for analyzing multiserver systems. *Queueing Systems* **2022**, *102*, 143–174. Preprint arXiv:2109.12663, <https://doi.org/10.1007/s11134-022-09848-6>.
43. Parekh, A.K.; Gallager, R.G. A generalized processor sharing approach to flow control in integrated services networks: the single-node case. *IEEE/ACM Transactions on Networking* **1993**, *1*, 344–357. <https://doi.org/10.1109/90.234856>.
44. Parekh, A.K.; Gallager, R.G. A generalized processor sharing approach to flow control in integrated services networks: the multiple node case. *IEEE/ACM Transactions on Networking* **1994**, *2*, 137–150. <https://doi.org/10.1109/90.298432>.
45. Bennett, J.C.R.; Benson, K.; Charny, A.; Courtney, W.F.; Le Boudec, J.Y. Delay Jitter Bounds and Packet Scale Rate Guarantee for Expedited Forwarding. *IEEE/ACM Transactions on Networking* **2002**, *10*, 529–540. <https://doi.org/10.1109/TNET.2002.801404>.
46. Bennett, J.C.R.; Zhang, H. WF²Q: Worst-case Fair Weighted Fair Queueing. In Proceedings of the IEEE INFOCOM '96, 1996, Vol. 1, pp. 120–128. <https://doi.org/10.1109/INFCOM.1996.497885>.
47. Stoica, I.; Abdel-Wahab, H. Earliest Eligible Virtual Deadline First: A Flexible and Accurate Mechanism for Proportional Share Resource Allocation. Technical Report TR-95-22, Department of Computer Science, Old Dominion University, Norfolk, Virginia, 1995. Revised 26 January 1996.
48. Graham, R.L. Bounds on Multiprocessing Timing Anomalies. *SIAM Journal on Applied Mathematics* **1969**, *17*, 416–429. <https://doi.org/10.1137/0117039>.
49. Kleinrock, L. A conservation law for a wide class of queueing disciplines. *Naval Research Logistics Quarterly* **1965**, *12*, 181–192. <https://doi.org/10.1002/nav.3800120206>.
50. Tsafir, D.; Etsion, Y.; Feitelson, D.G. Backfilling Using System-Generated Predictions Rather than User Runtime Estimates. *IEEE Transactions on Parallel and Distributed Systems* **2007**, *18*, 789–803. <https://doi.org/10.1109/TPDS.2007.70606>.
51. Gaussier, E.; Glesser, D.; Reis, V.; Trystram, D. Improving backfilling by using machine learning to predict running times. In Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '15), 2015, pp. 1–10. <https://doi.org/10.1145/2807591.2807646>.
52. Gaussier, E.; Lelong, J.; Reis, V.; Trystram, D. Online Tuning of EASY-Backfilling using Queue Reordering Policies. *IEEE Transactions on Parallel and Distributed Systems* **2018**, *29*, 2304–2316. <https://doi.org/10.1109/TPDS.2018.2820699>.
53. Zhang, E.; Wu, F.; Lu, X. An Intelligent Scheduling System for Large-Scale Online Judging. In Proceedings of the Computer Science and Education. Computer Science and Technology (ICCSE 2023). Springer Nature Singapore, 2024, Vol. 2023, *Communications in Computer and Information Science*, pp. 265–279. https://doi.org/10.1007/978-981-97-0730-0_24.
54. Fu, Y.; Zhu, S.; Su, R.; Qiao, A.; Stoica, I.; Zhang, H. Efficient LLM Scheduling by Learning to Rank, 2024, [2408.15792].
55. Li, Y.; Chen, Y.; Chen, J.; Choukse, E.; Qiu, H.; Suh, G.E.; Fonseca, R.; Scully, Z.; Gupta, U. Beyond Prediction: Tail-Aware Scheduling for LLM Inference, 2026, [arXiv:cs.LG/2606.18431]. <https://doi.org/10.48550/arXiv.2606.18431>.
56. Wu, B.; Zhong, Y.; Zhang, Z.; Liu, S.; Liu, F.; Sun, Y.; Huang, G.; Liu, X.; Jin, X. Fast Distributed Inference Serving for Large Language Models, 2023, [arXiv:cs.LG/2305.05920]. <https://doi.org/10.48550/arXiv.2305.05920>.
57. Luo, M.; Shi, X.; Cai, C.; Zhang, T.; Wong, J.; Wang, Y.; Wang, C.; Huang, Y.; Chen, Z.; Gonzalez, J.E.; et al. Autellix: An Efficient Serving Engine for LLM Agents as General Programs, 2025, [arXiv:cs.LG/2502.13965]. <https://doi.org/10.48550/arXiv.2502.13965>.
58. Fu, Y.; Liu, L.; Wang, H.; Cheng, Y.; Chen, S. SFS: Smart OS Scheduling for Serverless Functions. In Proceedings of the SC22: International Conference for High Performance Computing, Networking, Storage and Analysis. IEEE, 2022, pp. 1–16. Preprint arXiv:2209.01709, <https://doi.org/10.1109/SC41404.2022.00047>.
59. Fu, Y.; Shi, R.; Wang, H.; Chen, S.; Cheng, Y. ALPS: An Adaptive Learning, Priority OS Scheduler for Serverless Functions. In Proceedings of the 2024 USENIX Annual Technical Conference (USENIX ATC 24). USENIX Association, 2024, pp. 19–36.

60. Massey, W.A.; Whitt, W. Peak congestion in multi-server service systems with slowly varying arrival rates. *Queueing Systems* **1997**, *25*, 157–172. <https://doi.org/10.1023/A:1019156418862>.
61. Jennings, O.B.; Mandelbaum, A.; Massey, W.A.; Whitt, W. Server Staffing to Meet Time-Varying Demand. *Management Science* **1996**, *42*, 1383–1394. <https://doi.org/10.1287/mnsc.42.10.1383>.
62. Green, L.V.; Kolesar, P.J.; Whitt, W. Coping with Time-Varying Demand When Setting Staffing Requirements for a Service System. *Production and Operations Management* **2007**, *16*, 13–39. <https://doi.org/10.1111/j.1937-5956.2007.tb00164.x>.
63. Feldman, Z.; Mandelbaum, A.; Massey, W.A.; Whitt, W. Staffing of Time-Varying Queues to Achieve Time-Stable Performance. *Management Science* **2008**, *54*, 324–338. <https://doi.org/10.1287/mnsc.1070.0821>.
64. Harchol-Balter, M. *Performance Modeling and Design of Computer Systems: Queueing Theory in Action*; Cambridge University Press, 2013. <https://doi.org/10.1017/CBO9781139226424>.
65. Xu, K.; Hu, W.; Leskovec, J.; Jegelka, S. How Powerful are Graph Neural Networks? In Proceedings of the International Conference on Learning Representations (ICLR 2019), 2019. Preprint arXiv:1810.00826.
66. Robinson, J.; Ranjan, R.; Hu, W.; Huang, K.; Han, J.; Dobles, A.; Fey, M.; Lenssen, J.E.; Yuan, Y.; Zhang, Z.; et al. RelBench: A Benchmark for Deep Learning on Relational Databases. In Proceedings of the Advances in Neural Information Processing Systems 37, Datasets and Benchmarks Track (NeurIPS 2024), 2024. Preprint arXiv:2407.20060.
67. Gu, J.; Ranjan, R.; Kanatsoulis, C.; Tang, H.; Jurkovic, M.; Hudovernik, V.; Znidar, M.; Chaturvedi, P.; Shroff, P.; Li, F.; et al. RelBench v2: A Large-Scale Benchmark and Repository for Relational Data, 2026, [2602.12606].
68. Grinsztajn, L.; Oyallon, E.; Varoquaux, G. Why Do Tree-Based Models Still Outperform Deep Learning on Typical Tabular Data? In Proceedings of the Advances in Neural Information Processing Systems 35 (NeurIPS 2022), 2022, pp. 507–520. <https://doi.org/10.52202/068431-0037>.
69. Daley, D.J. Certain optimality properties of the first-come first-served discipline for G/G/s queues. *Stochastic Processes and their Applications* **1987**, *25*, 301–308. [https://doi.org/10.1016/0304-4149\(87\)90208-0](https://doi.org/10.1016/0304-4149(87)90208-0).
70. Jørgensen, B. Exponential Dispersion Models. *Journal of the Royal Statistical Society, Series B* **1987**, *49*, 127–145. <https://doi.org/10.1111/j.2517-6161.1987.tb01685.x>.
71. Yang, Y.; Qian, W.; Zou, H. Insurance Premium Prediction via Gradient Tree-Boosted Tweedie Compound Poisson Models. *Journal of Business & Economic Statistics* **2018**, *36*, 456–470. <https://doi.org/10.1080/07350015.2016.1200981>.
72. Duan, N. Smearing Estimate: A Nonparametric Retransformation Method. *Journal of the American Statistical Association* **1983**, *78*, 605–610. <https://doi.org/10.1080/01621459.1983.10478017>.
73. Kiefer, J.; Wolfowitz, J. On the theory of queues with many servers. *Transactions of the American Mathematical Society* **1955**, *78*, 1–18. <https://doi.org/10.1090/S0002-9947-1955-0066587-3>.
74. Cruz, R.L. A Calculus for Network Delay, Part I: Network Elements in Isolation. *IEEE Transactions on Information Theory* **1991**, *37*, 114–131. <https://doi.org/10.1109/18.61109>.
75. Le Boudec, J.Y.; Thiran, P. *Network Calculus: A Theory of Deterministic Queueing Systems for the Internet*; Vol. 2050, *Lecture Notes in Computer Science*, Springer: Berlin, Heidelberg, 2001. <https://doi.org/10.1007/3-540-45318-0>.
76. Grosz, I.; Scully, Z.; Harchol-Balter, M. SRPT for multiserver systems. *Performance Evaluation* **2018**, *127–128*, 154–175. Preprint arXiv:1805.07686, <https://doi.org/10.1016/j.peva.2018.10.001>.
77. Wolff, R.W. Upper bounds on work in system for multichannel queues. *Journal of Applied Probability* **1987**, *24*, 547–551. Also indexed as doi:10.2307/3214279, <https://doi.org/10.1017/S0021900200031193>.
78. Smith, W.E. Various optimizers for single-stage production. *Naval Research Logistics Quarterly* **1956**, *3*, 59–66. <https://doi.org/10.1002/nav.3800030106>.
79. Kuzilek, J.; Hlosta, M.; Zdrahal, Z. Open University Learning Analytics dataset. *Scientific Data* **2017**, *4*, 170171. <https://doi.org/10.1038/sdata.2017.171>.
80. Universidade Federal do Amazonas, Instituto de Computação. CodeBench. <https://codebench.icomp.ufam.edu.br/>. Accessed 2026-09-23.
81. Shahradd, M.; Fonseca, R.; Goiri, Í.; Chaudhry, G.; Batur, P.; Cooke, J.; Laureano, E.; Tresness, C.; Russinovich, M.; Bianchini, R. Serverless in the Wild: Characterizing and Optimizing the Serverless Workload at a Large Cloud Provider. In Proceedings of the 2020 USENIX Annual Technical Conference (USENIX ATC 20). USENIX Association, 2020, pp. 205–218.
82. Verma, A.; Pedrosa, L.; Korupolu, M.; Oppenheimer, D.; Tune, E.; Wilkes, J. Large-scale cluster management at Google with Borg. In Proceedings of the Tenth European Conference on Computer Systems (EuroSys '15), 2015, pp. 1–17. <https://doi.org/10.1145/2741948.2741964>.
83. Parallel Workloads Archive. The LPC Log. https://www.cs.huji.ac.il/labs/parallel/workload/l_lpc/. Parallel Workloads Archive; data provided by Emmanuel Medernach; accessed 2026-09-23.
84. Smith, D.R.; Whitt, W. Resource Sharing for Efficiency in Traffic Systems. *Bell System Technical Journal* **1981**, *60*, 39–55. <https://doi.org/10.1002/j.1538-7305.1981.tb00221.x>.

85. Feitelson, D.G. Resampling with Feedback: A New Paradigm of Using Workload Data for Performance Evaluation. In *Job Scheduling Strategies for Parallel Processing (JSSPP 2021)*; Springer, 2021; Vol. 12985, *Lecture Notes in Computer Science*, pp. 3–32. https://doi.org/10.1007/978-3-030-88224-2_1.
86. Shmueli, E.; Feitelson, D.G. On Simulation and Design of Parallel-Systems Schedulers: Are We Doing the Right Thing? *IEEE Transactions on Parallel and Distributed Systems* **2009**, *20*, 983–996. <https://doi.org/10.1109/TPDS.2008.152>.
87. Harchol-Balter, M.; Schroeder, B.; Bansal, N.; Agrawal, M. Size-based scheduling to improve web performance. *ACM Transactions on Computer Systems* **2003**, *21*, 207–233. <https://doi.org/10.1145/762483.762486>.
88. Harchol-Balter, M. Task assignment with unknown duration. *Journal of the ACM* **2002**, *49*, 260–288. Conference version: Proceedings of the 20th International Conference on Distributed Computing Systems (ICDCS 2000), pp. 214–224, doi:10.1109/ICDCS.2000.840932, <https://doi.org/10.1145/506147.506154>.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.